

DEPARTMENT OF COMPUTER SCIENCE

Moderne Softwarearchitektur

MSA26-PeerReview-Gruppe-A

Abgabedatum:

17.07.2026

Eingereicht von:

Matthias Matthies

Matr.-Nr.: 106971
stud106971@fh-wedel.de

Luca Jannsen

Matr.-Nr.: 107091
stud107091@fh-wedel.de

Marcel Ossig

Matr.-Nr.: 106970
stud106970@fh-wedel.de

Gideon Gyebi

Matr.-Nr.: 106968
stud106968@fh-wedel.de

Betreut von:

Prof. Dr. Ulrich Hoffmann

Fachhochschule Wedel
Feldstraße 143
22880 Wedel
Tel: 04103 - 8048 - 41
urlich.hoffmann@fh-wedel.de

Inhaltsverzeichnis

Abkürzungsverzeichnis	III
1 Einleitung	1
1.1 Kontext der Aufgabenstellung	1
1.2 Zielsetzung	2
1.3 Umsetzung und Werkzeugeinsatz	2
1.4 Aufgabenverteilung im Team	3
2 Features der App	4
2.1 Lastenheft	4
2.1.1 Konzept	4
2.1.1.1 Zielgruppen	4
2.1.1.2 Ziele der Nutzer	4
2.1.2 Funktionale Anforderungen	5
2.1.2.1 Autoren / Studierende	5
2.1.2.2 Lehrende	5
2.1.2.3 Reviewer	6
2.1.2.4 Prüfungsamt	6
2.1.2.5 System-Administration	6
2.1.3 Nichtfunktionale Anforderungen	7
2.1.3.1 Allgemeine Anforderungen	7
2.1.3.2 Technische Anforderungen	7
2.2 Out of scope	7
3 Architekturentwurf	9
3.1 Übersicht	9
3.2 Infrastruktur Architektur	11
3.2.1 Compute	12
3.2.2 Netzwerk	13
3.2.3 Datenbank Architektur	15
3.2.4 User Management	16
3.3 Configuration Service (Plugin Service)	17
3.4 Server-Sent Events (SSE) im Communication und Notification Service	17
3.5 Strategy Pattern im Notification Service	18
3.6 Web-UI	18
3.7 DevOps und Infrastructure as Code	19
4 Umsetzung und Workflow	21
4.1 Baseline	21
4.2 Agentic Coding	22
4.2.1 Agents.MD / Claude.MD	22

4.2.2	Antigravity	23
4.2.3	Claude Code	24
4.2.4	OpenCode	25
4.2.5	GitHub Copilot Reviewer	27
4.3	Integration von Continuous Integration (CI) und Continuous Deployment (CD)	27
4.4	Manuelles Testen	28
4.5	Automatisierte Testabdeckung	28
4.6	Optimierung	29
4.7	Dokumentation	31
5	Kritik und Einordnung des Vibecoding	32
5.1	Was lief gut	32
5.2	Was lief eher schlecht	32
5.3	Herausforderungen	33
5.4	Zukunfts-Ausrichtung	34
6	Kurze Zusammenfassung	36
6.1	Zusammenfassung	36
6.2	Fazit	36
6.3	Ausblick	37
A	Repository	39
	Literaturverzeichnis	40
	Eigenständigkeitserklärung	42

Abkürzungsverzeichnis

ALB – Application Load Balancer

AVP – Amazon Verified Permissions

AWS – Amazon Web Services

BFF – Backend for Frontend

CD – Continuous Deployment

CDK – AWS Cloud Development Kit

CDN – Content Delivery Network

CI – Continuous Integration

CRAP – Change Risk Anti-Patterns

ECR – Amazon Elastic Container Registry

ECS – Amazon Elastic Container Service

IAM – Identity and Access Management

IaC – Infrastructure as Code

LLM – Large Language Model

MVP – Minimum Viable Product

NAT – Network Address Translation

RBAC – Role-Based Access Control

S3 – Amazon Simple Storage Service

SPA – Single Page Application

SQS – Amazon Simple Queue Service

SSE – Server-Sent Events

VPC – Virtual Private Cloud

Multi-AZ – Multiple Availability Zones

1 Einleitung

Diese Ausarbeitung beschreibt die Konzeption und Umsetzung des Systems „PeerReview“, welches im Rahmen der Lehrveranstaltung Moderne Softwarearchitekturen im Sommersemester 2026 an der Fachhochschule Wedel entstanden ist. Im folgenden Kapitel werden zunächst der fachliche und organisatorische Kontext der Aufgabenstellung dargestellt, anschließend die daraus abgeleiteten Ziele des Projekts erläutert sowie das grundsätzliche Vorgehen bei der Umsetzung und der Einsatz KI-gestützter Werkzeuge skizziert. Abschließend wird die Aufteilung der Aufgaben innerhalb des vierköpfigen Teams dargestellt.

1.1 Kontext der Aufgabenstellung

Ausgangspunkt des Projekts ist die von Prof. Dr. Ulrich Hoffmann gestellte Aufgabe, ein webbasiertes System zu entwerfen und zu realisieren, das die gegenseitige Begutachtung wissenschaftlicher Arbeiten, etwa Seminar- oder Projektarbeiten, durch Studierende oder Mitarbeitende einer Organisation systematisch unterstützt. Es wurde dementsprechend ein PeerReview-System gefordert, welches die Einreichung von Arbeiten, die Zuweisung von Gutachtern, die Erfassung von Bewertungen und Kommentaren sowie die Aggregation und Rückgabe der Gutachten an die Einreichenden übernimmt.

Als denkbare Funktionen wurden in der Aufgabenstellung unter anderem die Einreichung von Dokumenten, eine regelbasierte oder manuelle Zuweisung von Gutachtern, strukturierte Bewertungsformulare, anonymisierte Begutachtung sowie eine Fristen- und Erinnerungsverwaltung genannt. Eine zentrale fachliche Anforderung besteht darin, über eine Plugin-Architektur unterschiedliche Begutachtungsworkflows wie Einfachblind, Doppelblind oder Open Review zu unterstützen, ohne dass die jeweilige Verfahrenslogik fest im Kern des Systems verankert wird. Die Aufgabenstellung legte hierbei einen starken Fokus auf die zugrunde liegende Architektur und warf explizit die Fragen auf, wie Workflow-Zustände zuverlässig verwaltet, Benachrichtigungen ereignisgesteuert ausgelöst und die einzelnen Komponenten geeignet skaliert werden können.

Darüber hinaus wurde der Einsatz von Werkzeugen der KI wie GitHub Copilot, Claude oder ChatGPT bei Entwurf, Implementierung, Test und Dokumentation ausdrücklich erwünscht, verbunden mit der Anforderung, den jeweiligen Einsatz sowie dessen Nutzen und Grenzen im Rahmen der Projektdokumentation zu reflektieren. Als weitere Vorgaben galten die Programmierung isolierter Komponenten samt zugehöriger Test-Clients, die gemeinsame Orchestrierung und Bereitstellung des Gesamtsystems auf einem Server sowie eine abschließende Präsentation der Ergebnisse.

1.2 Zielsetzung

Aus der in Abschnitt 1.1 beschriebenen Aufgabenstellung wurden für dieses Projekt konkrete Umsetzungsziele abgeleitet. Die geforderte Plugin-Fähigkeit sowie die fachliche Zerlegung des Begutachtungsprozesses sollten dabei nicht nur erfüllt, sondern durch eine konsequente Microservice-Architektur mit eigenständigen, lose gekoppelten Services technisch untermauert werden; wie diese Architektur im Detail aussieht, wird im Kapitel zum Architekturentwurf (Abschnitt 3) hergeleitet. Weiterhin sollte das gesamte System zunächst als Minimum Viable Product (MVP) realisiert werden, dessen Funktionsumfang gemeinsam im Team festgelegt und im weiteren Verlauf agil erweitert werden konnte.

Auf infrastruktureller Ebene wurde das Ziel verfolgt, sämtliche Komponenten vollständig über Infrastructure as Code (IaC) zu beschreiben und automatisiert in eine öffentlich erreichbare Cloud-Umgebung auf Basis von Amazon Web Services (AWS) auszurollen, ergänzt durch eine durchgängige Pipeline für Continuous Integration (CI) und Continuous Deployment (CD) für Build, Test und Deployment.

Neben diesen technischen Zielen verfolgte das Team ein übergeordnetes, eher experimentelles Ziel: den geforderten Einsatz von Werkzeugen der KI nicht nur pflichtgemäß umzusetzen, sondern bewusst zu untersuchen, in welchem Umfang sich eine verteilte Microservice-Architektur auf AWS mithilfe agentischer Coding-Werkzeuge realisieren lässt und wo die Grenzen eines solchen Vorgehens liegen. Diese Fragestellung wurde als fortlaufendes Experiment über die gesamte Projektlaufzeit begleitet. Die dabei gewonnenen Erkenntnisse werden in Abschnitt 4 beschrieben und in Abschnitt 5 kritisch eingeordnet.

1.3 Umsetzung und Werkzeugeinsatz

Das grundsätzliche Vorgehen bei der Realisierung der genannten Ziele gliederte sich in drei aufeinanderfolgende Schritte. Zunächst wurde die Architektur des Gesamtsystems gemeinsam im Team entworfen. Darauf aufbauend wurde eine technische Grundlage geschaffen, die den einzelnen Komponenten als gemeinsamer Ausgangspunkt diene. Erst im dritten Schritt wurden die fachlichen Anforderungen der einzelnen Services unter intensivem Einsatz agentischer Coding-Werkzeuge implementiert. Wie die einzelnen Komponenten fachlich zusammenwirken und wie die Architektur sowie die Infrastruktur im Detail umgesetzt wurden, wird im nachfolgenden Kapitel zu den Features der Anwendung sowie im Kapitel zum Architekturentwurf (Abschnitt 3) ausführlich beschrieben.

Bei der eigentlichen Entwicklung wurde in großem Umfang auf agentische Coding-Werkzeuge zurückgegriffen. Den Schwerpunkt bildeten hierbei Claude Code von Anthropic

sowie Google Antigravity, ergänzt um punktuelle Erprobungen von OpenCode für einzelne Services sowie den Einsatz von GitHub Copilot als automatisierte Code-Review-Instanz innerhalb des Pull-Request-Workflows. Die konkrete Vorgehensweise, die verwendeten Werkzeuge sowie die dabei gemachten Erfahrungen werden in Abschnitt 4 beschrieben.

1.4 Aufgabenverteilung im Team

Wir, Marcel Ossig, Luca Jannsen, Matthias Matthies und Gideon Gyebi, setzten das Projekt als vierköpfiges Team um. Marcel übernahm die Rolle des Projektleiters und Architekten und verantwortete maßgeblich den Architekturentwurf sowie den Aufbau der grundlegenden Infrastruktur auf AWS. Luca, Matthias und Gideon implementierten als Softwareentwickler jeweils eigene, ihnen zugeordnete Services innerhalb der beschriebenen Microservice-Architektur. Testen und Dokumentation verstanden wir als geteilte Verantwortung und führten sie begleitend zur eigentlichen Implementierung durch.

Tabelle 1 fasst die grobe Verteilung der Aufgabenbereiche innerhalb des Teams zusammen.

Aufgabenbereich	Verantwortlich
Projektleitung, Architekturentwurf und Infrastruktur auf AWS (inkl. initialer Baseline sowie CI und CD)	Marcel
Matching Service, Communication Service und User Service	Marcel
Configuration Service	Luca, Matthias
Submission Service	Matthias
Response Service und Notification Service	Gideon
Web-UI	Luca
Einrichtung für KI (<code>AGENTS.md</code> / <code>CLAUDE.md</code>) und Optimierung des Index Change Risk Anti-Patterns (CRAP)	Luca
Testen (u. a. Postman) und Dokumentation	Alle

Tabelle 1: Aufgabenverteilung im Team

Im folgenden Kapitel werden zunächst die Features des PeerReview-Systems aus Nutzersicht vorgestellt, bevor in den weiteren Kapiteln detailliert auf Architektur, Umsetzung und die kritische Reflexion des Projekts eingegangen wird.

2 Features der App

Die Applikation (das „PeerReview“-System) umfasst eine interaktive Web-Oberfläche sowie sieben dedizierte Backend-Services. Im Folgenden werden alle im aktuellen MVP implementierten Features aus Nutzersicht anhand eines einfachen Lastenhefts und der Nutzung von typischen Benutzergeschichten (User Stories) dargestellt.

2.1 Lastenheft

Das Lastenheft beschreibt das Grundkonzept des Systems sowie die daraus abgeleiteten funktionalen und nichtfunktionalen Anforderungen.

2.1.1 Konzept

Das Konzept legt die Zielgruppen des Systems und deren jeweilige Ziele als Grundlage für die nachfolgenden Anforderungen fest.

2.1.1.1 Zielgruppen

Das System unterscheidet fünf Zielgruppen mit jeweils eigenen Rollen und Berechtigungen.

Autoren / Studierende: Autoren bzw. Studierende nutzen das System als zentrale Plattform zur Abwicklung ihrer Einreichungen, wie beispielsweise Abschlussarbeiten oder Seminararbeiten.

Lehrende: Dozierende legen Abgaben für Autoren an, konfigurieren das gewünschte Begutachtungsverfahren sowie den Bewertungsbogen.

Reviewer: Reviewer begutachten ihnen zugewiesene Arbeiten, vergeben Bewertungen anhand des vorgegebenen Bewertungsbogens und beantworten Rückfragen der Autoren.

Prüfungsamt: Das Prüfungsamt nimmt eine administrative Rolle ein. Die Mitarbeiter pflegen den Gutachterpool, hinterlegen Fachgebiete und steuern Profile.

System-Administration: Die System-Administration ist für die technische Überwachung und globale Konfiguration zuständig. Sie verwaltet Profile, Berechtigungen und globale Themen-Tags und behält den Überblick über registrierte Review-Typen (Plugins) und aktive Templates.

2.1.1.2 Ziele der Nutzer

Autoren verfolgen das Ziel, ihre wissenschaftlichen Arbeiten fristgerecht einzureichen und nachvollziehbares Feedback zu erhalten. Lehrende möchten den Begutachtungsprozess für ihre Abgaben passend konfigurieren. Reviewer wollen ihnen zugewiesene

Arbeiten effizient und strukturiert begutachten können. Das Prüfungsamt verfolgt das Ziel, zentrale Abgaben sowie den Gutachterpool zu verwalten und jederzeit den Gesamtüberblick über laufende Verfahren zu behalten. Die System-Administration möchte Nutzer, Berechtigungen und Systemkomponenten zentral pflegen können.

2.1.2 Funktionale Anforderungen

Die funktionalen Anforderungen gliedern sich nach den fünf Zielgruppen und beschreiben die ihnen jeweils zur Verfügung stehenden Funktionen im MVP.

2.1.2.1 Autoren / Studierende

- **Abgaben selbstständig anlegen:** Erstellen eigener Arbeiten (z. B. Abschlussarbeiten) als Einzel- oder Gruppenarbeit unter Angabe von Mitautoren sowie (je nach Vorlage) Wunschprüfern und individuellen Fristen.
- **Wissenschaftliche Arbeiten einreichen:** Hochladen eines Dokuments im Format PDF zur konfigurierten Abgabe vor Ablauf der Abgabefrist.
- **Fristen und Status im Blick behalten:** Übersicht anstehender Fristen im Kalender und Live-Verfolgung des Bearbeitungsstands der eigenen Abgabe auf dem Dashboard.
- **Ergebnisse & Feedback abrufen:** Detaillierte Einsicht in Noten, Kommentare und ausgefüllte Kriterien nach Abschluss der Bewertung.
- **Fragen klären (Chat):** Austausch mit Gutachtern über den integrierten Chat (aktiviert bei Open Review, deaktiviert bei Doppelblind / Einfach-Blind zur Wahrung der Anonymität).
- **Echtzeit-Benachrichtigungen:** Erhalt von In-App-Meldungen bei Statusänderungen der eigenen Abgabe (z. B. nach erfolgreicher Erstellung oder Vorliegen einer Bewertung).
- **Gutachten durch KI anfordern:** Ein durch KI generiertes Gutachten kann zur eigenen Arbeit angefordert werden.

2.1.2.2 Lehrende

- **Abgaben anlegen und konfigurieren:** Erstellen neuer Arbeiten mit individuellen Titeln, Deadlines, Mitautoren und Themen-Tags, die von Autoren einzureichen sind.
- **Review-Verfahren & Kriterien bestimmen:** Auswahl des Review-Prozesses (Doppelblind, Einfach-Blind, Open Review) sowie Auswahl des Bewertungsbogens über die Art der Abgabe.
- **Automatische oder manuelle Definition des Reviewers:** Manuelle Auswahl eines Reviewers (sich selbst, wenn eine anschließende Einsicht über den Status erwünscht ist) oder Verzicht auf eine Auswahl für das automatische Matching.

2.1.2.3 Reviewer

- **Zugewiesene Arbeiten einsehen:** Übersicht über alle Einreichungen, bei denen man als Gutachter eingeteilt ist.
- **Abgaben begutachten:** Download der eingereichten Arbeiten im Format PDF zur Korrektur.
- **Bewertungen abgeben:** Ausfüllen des vom Lehrenden vorgegebenen Bewertungsbogens im System mit Noten und textuellem Gesamt-Feedback.
- **Direktkommunikation (Chat):** Austausch mit Autoren über einen integrierten Chat (aktiviert bei Open Review, deaktiviert bei Doppelblind / Einfach-Blind zum Schutz der Anonymität).
- **Echtzeit-Benachrichtigungen:** Erhalt von In-App-Meldungen bei neu zugewiesenen Arbeiten zur Begutachtung.
- **Gutachten durch KI anfordern:** Ein durch KI generiertes Gutachten kann als Unterstützung bei der Begutachtung angefordert werden.

2.1.2.4 Prüfungsamt

- **Zentrale Abgaben konfigurieren:** Erstellen offizieller Abgaben mit globalen Fristen und Bewertungsbögen für Studierende.
- **Gutachterpool pflegen:** Registrierung von Prüfern im System inklusive Hinterlegen von Fachgebieten (Themen-Tags) und Aktivieren/Deaktivieren von Profilen.
- **Überwachung des Gesamtprozesses:** Uneingeschränkte Sicht auf den Status aller Abgaben, automatischen Zuweisungen, Deadlines und Korrekturfortschritte im System.
- **Systemstatistiken einsehen:** Übersicht über globale Kennzahlen wie Benutzerzahlen, aktive Plugins und Templates im Admin-Bereich.
- **Benutzer und Rollen verwalten:** Zuweisung von Berechtigungsgruppen (z. B. Lehrender, Reviewer, Autor, Prüfungsamt) für registrierte Benutzer.

2.1.2.5 System-Administration

- **Globale Benutzerverwaltung:** Umfassende Verwaltung aller Profile und Systemberechtigungen.
- **Systemkomponenten einsehen:** Übersicht über registrierte Review-Typen (Plugins) und aktive Templates sowie Verwaltung globaler Themen-Tags.

Für technische Störungen oder Zuweisungskonflikte existiert im MVP keine dedizierte, korrigierende Eingriffsmöglichkeit über die Anwendung. Weder Lehrende noch die System-Administration können fehlerhafte automatische Zuweisungen manuell übersteuern (siehe Abschnitt 2.2).

2.1.3 Nichtfunktionale Anforderungen

Die nichtfunktionalen Anforderungen umfassen allgemeine Qualitätsmerkmale der Anwendung sowie technische Rahmenbedingungen der zugrunde liegenden Infrastruktur.

2.1.3.1 Allgemeine Anforderungen

- **Rollenbasierte Oberfläche:** Jede der fünf Zielgruppen erhält ein auf ihre Aufgaben zugeschnittenes Dashboard mit den jeweils relevanten Funktionen.
- **Anonymität:** Bei Doppelblind- und Einfach-Blind-Verfahren müssen identifizierende Informationen (Namen, Chat) für die jeweils andere Partei zuverlässig verborgen bleiben.
- **Echtzeit-Rückmeldung:** Statusänderungen (neue Zuweisung, abgeschlossene Bewertung) müssen den betroffenen Nutzern zeitnah als In-App-Benachrichtigung angezeigt werden.
- **Bedienbarkeit:** Die Web-Oberfläche muss ohne Schulung durch die genannten Zielgruppen bedienbar sein.

2.1.3.2 Technische Anforderungen

- **Microservice-Architektur:** Die sieben Backend-Services sind eigenständig deploybar und kommunizieren synchron über REST sowie asynchron über Amazon Simple Queue Service (SQS) (siehe Abschnitt 3).
- **Database-per-Service:** Jeder Service verwaltet seine eigene, isolierte Datenbank; ein Datenaustausch zwischen Services erfolgt ausschließlich über APIs oder SQS, nicht über direkte Datenbankzugriffe.
- **Dokumentenablage:** Eingereichte Dokumente im Format PDF werden als Objekte in Amazon Simple Storage Service (S3) gespeichert.
- **Plugin-Architektur im Workflow Service (nach Refactoring im Configuration Service):** Unterschiedliche Begutachtungsverfahren (Doppelblind, Einfach-Blind, Open Review) und Bewertungsverfahren müssen als austauschbare Plugins ohne Änderung der Kernlogik unterstützt werden.
- **Cloud-Infrastruktur:** Betrieb auf Amazon Elastic Container Service (ECS) Fargate (ARM64 / Graviton) mit IaC über AWS Cloud Development Kit (CDK).
- **Skalierbarkeit & Kostenoptimierung:** Die Services werden per Scheduled Application Auto Scaling an Werktagen zwischen 16:00 und 22:00 UTC+2 und am Wochenende zwischen 11:00 und 23:00 UTC+2 hochskaliert; außerhalb dieser Zeiten laufen sie mit 0 Instanzen, um Betriebskosten zu minimieren.

2.2 Out of scope

Für das MVP wurde sich bewusst gegen die Implementierung bestimmter Features entschieden oder diese wurden zugunsten eines reduzierten Scopes zurückgestellt:

- **Korrigieren Fehlerhafter Zuweisungen:** Es gibt keine Funktion, mit der Prüfungsamt, Lehrende oder System-Administration fehlerhafte Zuweisungen nachträglich korrigieren können.
- **Annotationen in Dokumenten im Format PDF:** Direkte visuelle Markierungen, Kommentare oder Zeichnungen auf den Dokumenten im Format PDF im Browser wurden nicht implementiert. Die Begutachtung erfolgt stattdessen strukturiert über Bewertungsbögen und textuelles Gesamt-Feedback. (Begründung: Hohe Komplexität der UI bei geringem Mehrwert für das MVP).
- **Dateiformate abseits von PDF:** Es wird ausschließlich das `.pdf`-Format für Einreichungen unterstützt. Andere in der Aufgabenstellung denkbare Formate (z. B. `.zip`-Dateien für Quellcode-Abgaben) wurden nicht realisiert. Die Ausweitung auf weitere Datei-Formate ist zu einem möglichen späteren Zeitpunkt einfach umzusetzen.
- **Erinnerungsverwaltung (Reminder Service):** Es gibt kein automatisiertes System, das Benutzer vor Ablauf von Deadlines (Abgabe- oder Review-Fristen) benachrichtigt oder an ausstehende Aufgaben erinnert.
- **Externe Schnittstelle zur Anwendungsprogrammierung (z. B. zur Anbindung an ein LMS):** Eine Einbindung in externe Systeme wie Lernmanagementsysteme (z. B. Moodle, Canvas über LTI) wurde nicht umgesetzt. Die Endpunkte stehen jedoch zur Verfügung.
- **Dedizierter Statistik- & Reporting-Service:** Statistische Auswertungen sind auf einfache Kennzahlen wie KPI im Admin-Dashboard (Nutzerzahlen, aktive Plugins) beschränkt. Ein eigenständiger Dienst zur Aggregation komplexer Kennzahlen (z. B. Notenverläufe) wurde nicht umgesetzt.
- **Lokales Einzelkommando-Deployment:** Statt eines lokalen Aufbaus per `docker compose up` wird das System entweder vollautomatisiert über eine CI/CD-Pipeline (GitHub Actions) oder manuell via `docker build` und `docker push` ins ECR geladen und anschließend per `cdk deploy` auf AWS bereitgestellt. (Begründung: Durchgängiges Cloud-natives Deployment mit CD statt lokalem Docker-Compose-Stack; das System ist dauerhaft über eine öffentliche URL erreichbar.)

3 Architekturentwurf

Einen Kernpunkt dieser Ausarbeitung bildet die zugrunde liegende Softwarearchitektur. Um eine ausfallsichere und skalierbare Software zu entwickeln, fanden durchgehend moderne Architekturmuster Anwendung. Diese Muster wurden auf allen Ebenen des Systems berücksichtigt, von der grundlegenden Klassifizierung als Microservices-Architektur bis hin zur Nutzung spezifischer Webprotokolle wie Server-Sent Events (SSE) für die asynchrone Kommunikation. Im Folgenden werden die verschiedenen Architekturebenen detailliert dargestellt sowie die zugrunde liegenden Motivationen und die daraus resultierenden technischen Einschränkungen beleuchtet.

3.1 Übersicht

Die entwickelte Software implementiert eine Microservices-Architektur, bei der domänenspezifische Fachlichkeiten nach dem Prinzip der losen Kopplung gekapselt und in eigenständige Services ausgelagert wurden. Grundlage hierfür war eine detaillierte Analyse des gesamten Peer-Review-Prozesses, aus der sich spezifische fachliche Schwerpunkte (Bounded Contexts) ableiten ließen.

Insgesamt wurden acht Domänen identifiziert. Abbildung 1 bietet einen Überblick über alle Services sowie deren Relationen zueinander. Vier dieser Services bilden das Kern-Framework des eigentlichen Review-Prozesses, während vier weitere Services zusätzliche allgemeine oder technisch bedingte Domänen abdecken.

Die Services des Frameworks umfassen den Configuration Service, Matching Service, Submission Service und Response Service. Die Architektur des Erstgenannten wird im nachfolgenden Abschnitt 3.3 genau erläutert. Grundsätzlich besteht dessen Fachlichkeit darin, verschiedene Einreichungstypen sowie Review-Formate bereitzustellen und das Erstellen neuer Abgaben zu verwalten. Nach dem erfolgreichen Anlegen einer Abgabe ermittelt der Matching Service eine passende Kombination aus Abgabe und Reviewer, wobei die allgemeine Verfügbarkeit sowie die jeweiligen Fachgebiete berücksichtigt werden. Anschließend ermöglicht der Submission Service die eigentliche Abgabe der Arbeit durch den Autor mittels eines Uploads im Format PDF. Abschließend nimmt der Response Service die Gutachten der Reviewer entgegen und stellt diese nach Abschluss des gesamten Prozesses dem Autor zur Verfügung.

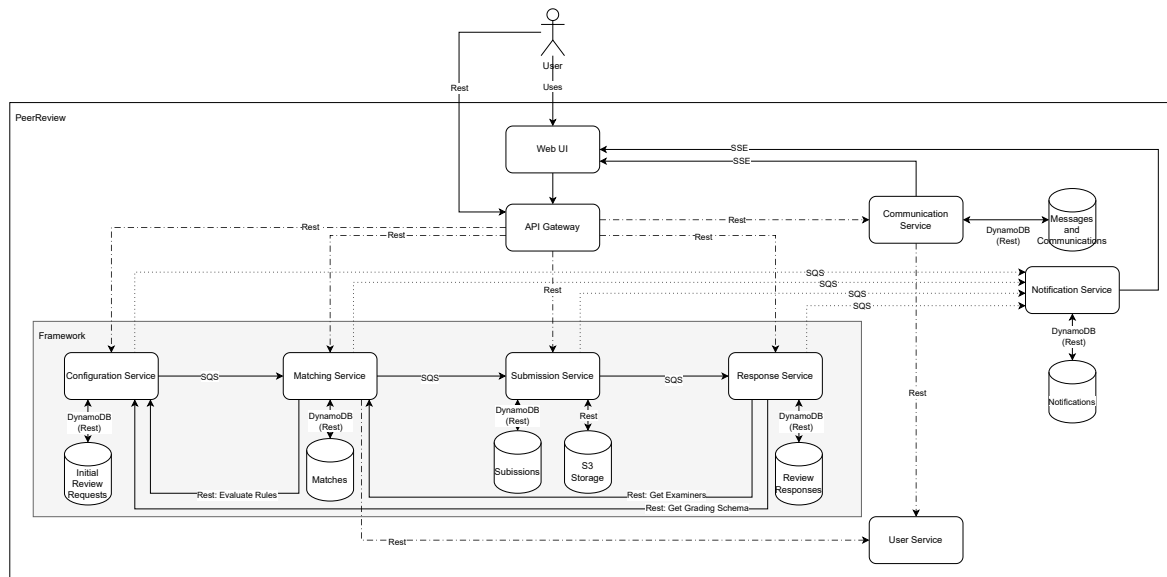


Abbildung 1: Applikationsarchitektur

Auffällig ist der Verzicht auf einen zentralen Orchestrations-Service (Orchestrator), welcher in Microservices-Architekturen häufig zum Einsatz kommt. Diese Entscheidung wird durch die Natur des Peer-Review-Verfahrens begünstigt: Es handelt sich um einen deterministischen Workflow, bei dem nach jedem abgeschlossenen Schritt eindeutig der nächste folgt. Die einzelnen Services des Frameworks können somit über eine ereignisgesteuerte Architektur (Event-Driven Architecture) mittels asynchroner Message-Queues direkt miteinander kommunizieren.

Die zusätzlichen Services umfassen den Communication Service, Notification Service, User Service sowie die Web-UI. Der Communication Service ermöglicht den interaktiven Austausch der Nutzer über die Plattform, entweder in Form von Direktnachrichten zwischen zwei Personen oder als fachlicher Austausch über eine spezifische Abgabe mit mehreren Beteiligten. Diese Funktion berücksichtigt strikt die Vorgaben des Configuration Service, der je nach Review-Format (z. B. Double-Blind-Verfahren) jegliche direkte Kommunikation unterbinden kann. Der Notification Service ist für die Benachrichtigung der Nutzer verantwortlich. Diese Benachrichtigungen werden vom System basierend auf spezifischen Ereignissen im Review-Prozess generiert und dem Anwender asynchron ausgespielt. Details dazu, wie die Software verschiedene Benachrichtigungskanäle bereitstellen kann, werden in Abschnitt 3.5 erläutert. Der User Service bietet eine einheitliche Schnittstelle für das Identity and Access Management (IAM), wobei dieser Service eine Abstraktionsschicht („Wrapper“) für Cognito auf AWS darstellt, was im Abschnitt 3.2.4 detailliert beschrieben wird. Die Web-UI bündelt die Schnittstellen der Backend-Services für den Endnutzer und wird in Abschnitt 3.6 weiter erläutert.

Die technologische Basis bilden im Backend Java 25 mit dem Spring Boot Framework sowie React mit Vite für das Frontend. Die Infrastruktur wird vollständig über

AWS abgebildet und im Abschnitt 3.2 näher beleuchtet. Der gewählte Microservices-Ansatz erlaubt prinzipiell eine polyglotte Entwicklung, bei der beliebige Technologien eingesetzt werden können, sofern die Schnittstellen zur Anwendungsprogrammierung (API) kompatibel bleiben. Zu diesem Zweck stellen alle Backend-Services ihre Funktionalitäten über vollständig dokumentierte Schnittstellen gemäß REST auf Basis von Spezifikationen gemäß OpenAPI bereit.

Die Backend-Services lassen sich als funktionale Basis-Services kategorisieren. Sie umfassen bewusst eng gefasste Fachlichkeiten und bilden isoliert betrachtet keinen eigenständigen, vollständigen Geschäftsprozess ab. Erst durch die Integration in der zustandslosen Web-UI, die als aggregierender Service agiert, verschmelzen die isolierten Schnittstellen zur Anwendungsprogrammierung zu einem zusammenhängenden Prozess. Dieser Ansatz der clientseitigen Bündelung bietet den Architekturvorteil, dass die Web-UI hochgradig austauschbar ist. Es können jederzeit neue Frontends entwickelt werden, die basierend auf den vorhandenen Schnittstellen gemäß REST einen modifizierten Detaillierungsgrad oder alternative User Journeys abbilden, ohne die Backend-Logik anzupassen.

3.2 Infrastruktur Architektur

Die Architektur basiert auf containerisierten Deployments der einzelnen Microservices. Die gesamte Bereitstellung der Compute-Ressourcen sowie weiterer Plattform-Dienste erfolgt über AWS. Es ist aus architektonischer Sicht kritisch anzumerken, dass die tiefe Integration proprietärer AWS-Services das Risiko eines Vendor-Lock-ins birgt, was für einen dauerhaften Produktivbetrieb sorgfältig evaluiert werden müsste. Im Rahmen dieser Arbeit entsprach ein Deployment in AWS sowie die exklusive Bindung an das entsprechende Ökosystem jedoch den expliziten Vorgaben der Aufgabenstellung und der durch die Hochschule bereitgestellten Infrastruktur.

Ein maßgebliches Entwurfskriterium war eine strikte Kosteneffizienz, weshalb ausschließlich moderne Serverless-Komponenten zum Einsatz kommen. Zugunsten der Kostenoptimierung wurde auf eine ausgedehnte High-Availability-Architektur mit Multiple Availability Zones (Multi-AZ) vorerst verzichtet, eine nachträgliche Implementierung ist jedoch möglich und vorbereitet.

Abbildung 2 veranschaulicht die eingesetzten Dienste von AWS sowie deren strukturelles Zusammenwirken innerhalb der Software. Die genaue Funktionsweise dieser Komponenten wird in den nachfolgenden Abschnitten detailliert beschrieben.

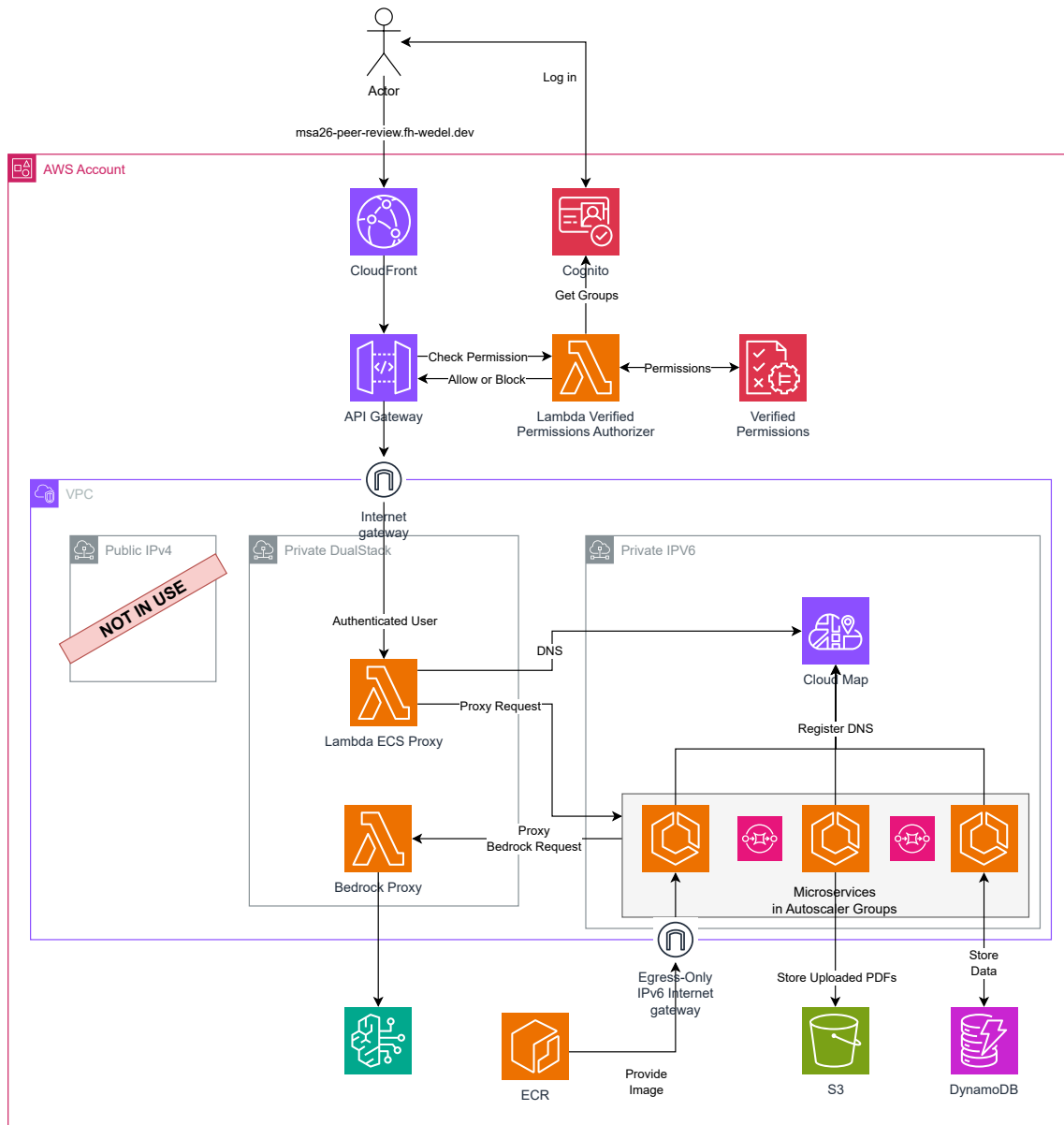


Abbildung 2: Infrastrukturarchitektur (Amazon Web Services (AWS))

3.2.1 Compute

Die Bereitstellung der Rechenleistung (Compute) erfolgt über ECS in Kombination mit der Serverless-Compute-Engine Fargate auf AWS. Fargate ermöglicht die vollautomatisierte Provisionierung von Laufzeitumgebungen für Docker-Container, deren Images in Amazon Elastic Container Registry (ECR) versioniert vorliegen.

Die Ausfallsicherheit der Services ist systembedingt hoch und kann durch horizontales Skalieren weiter optimiert werden. Zur Bewältigung von Lastspitzen kommt ein Autoscaler zum Einsatz, der mittels einer Target-Tracking-Skalierungsrichtlinie (Target Tracking Policy) eine durchschnittliche Auslastung der CPU von 75 % anstrebt. Bei Überschreiten dieses Schwellenwerts wird horizontal hochskaliert, bei geringerer Last

entsprechend herunterskaliert. Auch das kontinuierliche Ausrollen neuer Software-Versionen wird durch Fargate auf ECS nativ unterstützt, was moderne Deployment-Strategien wie Blue/Green-, Canary- oder Rolling-Deployments ermöglicht. Aktuell wird das Rolling-Deployment verwendet, wobei neue Service-Instanzen parallel gestartet werden und erst nach erfolgreicher Gesundheitsprüfung (Health Check) für den produktiven Datenverkehr freigegeben. Die alten Instanzen werden vollautomatisch deprovisioniert.

Zur weiteren Kostenreduktion nutzen alle auf ECS bereitgestellten Services eine Prozessorarchitektur gemäß ARM64 (Graviton von AWS), welche ein besseres Preis-Leistungs-Verhältnis aufweist als vergleichbare x86-Architekturen. Zudem werden Spot-Instanzen von AWS verwendet, die erhebliche Rabatte bieten, jedoch bei Kapazitätsengpässen kurzfristig terminiert werden können. Für den aktuellen Projektstatus ist dies ein idealer Kompromiss, da kurze Downtimes tolerierbar sind und der Autoscaler ausfallende Instanzen automatisch durch neue ersetzt. Für einen ausfallsicheren Produktivbetrieb könnte das System auf eine Strategie mit Multi-AZ erweitert werden, um den Ausfall einzelner Rechenzentren transparent zu kompensieren. Aufgrund der damit verbundenen Mehrkosten wurde dies jedoch im aktuellen Scope nicht umgesetzt.

Da jede Instanz mit der kleinstmöglichen Konfiguration von 0,256 vCPUs und 512 MB RAM dimensioniert ist, belaufen sich die reinen Compute-Kosten, unter der Annahme, dass kein Autoscaling stattfindet und kontinuierlich nur eine Instanz läuft, auf 0,078192 € pro Tag. In Summe resultiert dies in Kosten von 0,625536 € pro Tag für alle Services, was die Vorgabe einer hocheffizienten Infrastruktur vollumfänglich erfüllt.

Die Compute-Instanzen kommunizieren größtenteils asynchron und entkoppelt über SQS. Hierbei wurde das Paradigma der Data Ownership strikt angewendet: Jeder Service stellt eine dedizierte Queue von SQS als Eingangskanal (Input Queue) bereit. Andere Services, die Daten an diesen Service übergeben möchten, schreiben ihre Events direkt in die jeweilige Ziel-Queue.

Bestimmte fachliche Abläufe erfordern jedoch eine synchrone Kommunikation, welche direkt über REST abgebildet wurde. Dafür nutzen die Services interne Namen des DNS zur Service Discovery über CloudMap, deren Funktionsweise im folgenden Abschnitt zum Netzwerk erläutert wird.

3.2.2 Netzwerk

Netzwerkseitig wird die Software in einer Virtual Private Cloud (VPC) überwiegend in einem sicheren, privaten Subnetz bereitgestellt. Die Netzwerkkonfiguration ist ein kritischer Aspekt der Kostenoptimierung, da insbesondere passiv bereitgestellte Netzwerkinfrastruktur in der Cloud von AWS erhebliche Fixkosten verursachen kann.

Durch das Deployment in ein privates Subnetz sind die Instanzen nicht über das öffentliche Internet routingfähig, was die Sicherheit der Architektur signifikant erhöht. Dies bedeutet jedoch auch, dass die auf ECS bereitgestellten Instanzen ohne weitere Maßnahmen nicht nach außen kommunizieren können, um beispielsweise initiale Docker-Images aus ECR zu pullen. Ein klassischer Lösungsansatz bestünde in der Provisionierung eines von AWS verwalteten Network Address Translation (NAT) Gateway. Ein NAT Gateway verursacht jedoch hohe passive Kosten von ca. 50 € pro Monat, zuzüglich variabler Traffic-Kosten. Eine Alternative wäre die Zuweisung öffentlicher Adressen nach IPv4 an jede Instanz auf ECS. Da AWS jedoch Gebühren für die Nutzung öffentlicher Adressen nach IPv4 erhebt (ca. 3,50 € pro IP/Monat), würden sich die monatlichen Kosten bei acht Services auf knapp 30 € summieren.

Die implementierte Lösung nutzt stattdessen ein reines Netzwerk auf Basis von IPv6. AWS ermöglicht es, einem Subnetz auf Basis von IPv6 ein kostenfreies Egress-Only-Internet-Gateway zuzuordnen. Dadurch können die Instanzen auf ECS ausgehende Verbindungen über IPv6 aufbauen, um Abhängigkeiten oder Images aus ECR herunterzuladen. Obwohl AWS auch Dual-Stack-Subnetze (parallele Nutzung von IPv4 und IPv6) anbietet, war dieser Ansatz nicht zielführend. In einem Dual-Stack-Setup nutzen Instanzen auf ECS den Image-Pull über IPv4, was ohne NAT Gateway oder öffentliche Adresse nach IPv4 zwangsläufig fehlschlägt. Der konsequente Einsatz eines reinen Subnetzes auf Basis von IPv6 löst dieses Problem kostenneutral, schafft jedoch neue Herausforderungen beim Routing des eingehenden Datenverkehrs (Ingress).

Externe Requests müssen die Instanzen auf ECS erreichen, optimalerweise nicht über Adressen nach IPv6, sondern über DNS. Hierfür kommt das API Gateway von AWS zum Einsatz, das eine öffentlich verfügbare Auflösung über DNS ohne Fixkosten bereitstellt. Die technische Limitierung besteht jedoch darin, dass das API Gateway Instanzen auf ECS in einem reinen Netzwerk auf Basis von IPv6 nicht direkt als Ziel (Target) adressieren kann, da es zwingend Endpunkte nach IPv4 erwartet. Als Lösung wurden Lambda-Funktionen von AWS als Proxy-Schicht entwickelt und in einem separaten, Dual-Stack-fähigen Subnetz bereitgestellt. Das API Gateway routet eingehende Anfragen an diese Lambda-Proxies, welche die Requests an die Services auf ECS im Subnetz auf Basis von IPv6 weiterleiten. Die Lambdas können in einem Dual-Stack-Subnetz betrieben werden, da sie keine externen Docker-Images über IPv4 beziehen müssen und die Netzwerkstrecke nach IPv4 zum API Gateway systemseitig gewährleistet ist.

Für dieses Proxy-Pattern müssen die Lambda-Funktionen die internen, dynamischen Adressen nach IPv6 der Services auf ECS kennen. Zur Realisierung der Service Discovery kommt Cloud Map von AWS zum Einsatz. Alle Services auf ECS registrieren sich beim Bootvorgang vollautomatisch bei Cloud Map. Die Lambda-Proxies fragen den

internen Namen des DNS über Cloud Map ab, erhalten die aktuelle Adresse nach IPv6 des Ziels und leiten den Request entsprechend weiter.

Dieser Lambda-Proxy-Ansatz kann durch sogenannte Kaltstarts (Cold Starts) zeitweise zu erhöhten Latenzen führen, da die Initialisierung der Lambda-Laufzeitumgebung einige Sekunden beanspruchen kann. Eine performantere, aber kostenintensive Lösung ist die Nutzung von Provisioned Concurrency, bei der vorkonfigurierte Lambda-Instanzen warmgehalten werden. Diese Funktion wird im Projekt jedoch nur temporär zu Demonstrations- und Testzwecken aktiviert. Alternativ ließe sich ein Application Load Balancer (ALB) implementieren, der die Lambda-Proxies oder gar das API Gateway ersetzt, eine feste Adresse nach IPv4 bereitstellt und Lastverteilungs-Metriken nativ unterstützt. Die fixen Vorhaltekosten eines ALB belaufen sich jedoch auf ca. 16 € pro Monat. Bei einer dedizierten Bereitstellung pro Service würden somit monatliche Fixkosten von rund 130 € entstehen, was den ökonomischen Vorgaben des Projekts widerspricht.

Ein weiterer Lambda-Proxy wurde zwischen dem Response Service und Bedrock von AWS implementiert, da Bedrock derzeit keine nativen Verbindungen über IPv6 unterstützt. Durch das Proxying über eine Dual-Stack-fähige Lambda-Funktion konnte diese Einschränkung erfolgreich umgangen werden.

Den Abschluss der Netzwerkarchitektur bildet CloudFront von AWS als Content Delivery Network (CDN). CloudFront reduziert die globale Latenz durch Edge-Caching und leitet Anfragen effizient an den geografisch nächsten Edge-Standort weiter. Über CloudFront wurde zudem die Custom Domain `fh-wedel.dev` konfiguriert. Anstatt die flüchtigen Endpunkte des API Gateways direkt anzusprechen, ermöglicht Route 53 von AWS eine feste Auflösung über DNS, sodass externe Anfragen konsistent über die Custom Domain an das API Gateway geroutet werden.

3.2.3 Datenbank Architektur

Die Datenbankarchitektur unterliegt denselben strengen Vorgaben zur Kosteneffizienz. Es war zwingend erforderlich, eine vollständig serverlose Datenbanklösung ohne Fixkosten zu implementieren. Da die Geschäftsdaten keine strenge relationale Modellierung erforderten, fiel die Wahl auf DynamoDB von AWS, eine hochskalierbare Key-Value-Datenbank mit einem Pay-per-Request-Preismodell.

Um eine starke strukturelle Kopplung zu vermeiden und die Unabhängigkeit der Microservices zu wahren, wurde das Database-per-Service-Pattern konsequent umgesetzt. Jeder Service, der Persistenz benötigt, verfügt über eine eigene, isolierte DynamoDB-Tabelle.

Bei der Datenmodellierung kam konsequent ein modernes Single-Table-Design zum Einsatz. Dieser Ansatz ermöglicht es, alle zu einer Abgabe gehörenden Entitäten eines Services über eine einzige Query hocheffizient abzufragen. Komplizierte Joins über mehrere Tabellen hinweg oder das Absetzen sequenzieller Abfragen (N+1-Problem) entfallen dadurch vollständig.

Zudem wird das Prinzip der Datenhoheit (Data Sovereignty) strikt eingehalten: Jeder Service speichert ausschließlich die Daten, die in seinen Bounded Context fallen. Dies verhindert Datenredundanz über mehrere Services hinweg und eliminiert das Risiko von Dateninkonsistenzen bei Updates. Benötigt ein Service aggregierte Daten, fragt er diese zur Laufzeit über die REST-Schnittstelle des zuständigen Data Owners an.

3.2.4 User Management

Für das IAM werden Cognito von AWS und Amazon Verified Permissions (AVP) genutzt. Cognito agiert als Managed IdP und übernimmt komplexe Aufgaben wie die Bereitstellung gehosteter Login-Seiten, das Management von Anmeldeinformationen (Credentials) sowie die Verwaltung von Session-Tokens. Dies entspricht Best Practices der modernen Softwareentwicklung, da sich das Entwicklungsteam auf die Kernkompetenzen und die Geschäftslogik konzentrieren kann. Zudem ermöglicht Cognito die Gruppierung von Nutzern, was die Grundlage für eine rollenbasierte Zugriffskontrolle (Role-Based Access Control (RBAC)) bildet.

Zur Autorisierung der Endpunkte des API Gateways kommt AVP zum Einsatz. AVP definiert feingranulare Policies, die den Zugriff auf Ressourcen regulieren, sodass nur berechnete Cognito-Benutzergruppen bestimmte Endpunkte der API aufrufen dürfen. Die Durchsetzung dieser Richtlinien erfolgt über einen Lambda Authorizer, der vom API Gateway bei jedem eingehenden Request aufgerufen wird. Diese Lambda-Funktion validiert das Session-Token, gleicht die Cognito-Gruppen des Nutzers mit den in AVP hinterlegten Policies ab und trifft die finale Autorisierungsentscheidung (Allow/Deny).

Auch diese Architektur erzeugt keine passiven Kosten, und die nutzungsbasierten Gebühren für Cognito und AVP sind marginal. Der einzige Trade-off besteht in potenziellen Latenzen durch Kaltstarts des Lambda Authorizers. Auch hier kann durch Provisioned Concurrency die Antwortzeit signifikant optimiert werden, ein Effekt, der sich potenziert, wenn parallel auch der Lambda-Proxy vorkonfiguriert ist.

Zusätzliche Sicherheitsmechanismen sind direkt in der Geschäftslogik (Applikationsschicht) verankert. Hierbei entscheidet die Software kontextbezogen, ob ein Nutzer bestimmte Datensätze lesen oder mutieren darf (z. B. Author einer Abgabe). Diese Logik wird über Spring Security abgebildet, wobei die aus Cognito von AWS injizierten

Benutzernamen und Rollen als vertrauenswürdige Quelle (Single Source of Truth) dienen.

3.3 Configuration Service (Plugin Service)

Die Aufgabe des Configuration Service besteht darin, die Erstellung neuer Review-Abgaben zu verwalten und die hierbei verfügbaren Konfigurationsmöglichkeiten zur Abfrage bereitzustellen. Hierfür unterstützt der Service Plugins, welche es erlauben neue Review-Typen (z. B. Einfachblind oder Doppelblind) und -Formate zu definieren (z. B. Gruppenarbeit oder Bachelor Thesis). Hierdurch können beispielsweise unterschiedliche Bewertungskriterien in Abhängigkeit vom Review-Format spezifiziert werden. Der Service lädt die bereitgestellten Plugins beim Starten der Anwendung dynamisch, wodurch es nicht erforderlich ist, den Service bei der Einführung neuer Plugins neu zu bauen.

Der Service stellt diverse REST-Endpunkte zur Verfügung, über die das Frontend beim initialen Aufruf der PeerReview-Webseite unter anderem die verfügbaren Plugins ermittelt. Sobald dies geschehen ist, kann der Nutzer eine neue Abgabe erstellen, wobei der Titel der Arbeit, die Autoren, die zugehörige Fachrichtung sowie der Review-Typ und das Review-Format spezifiziert werden muss. In Abhängigkeit von der Rolle des Nutzers (z. B. Author, Teacher oder Admin) können auch weitere optionale Parameter wie die Abgabefrist oder vorgegebene Reviewer ausgewählt werden. Sobald die Abgabe beim Configuration Service eingereicht wurde, validiert der Service die Eingaben und speichert die Abgabe in einer DynamoDB-Tabelle. Anschließend wird ein Event in die SQS-Queue vom Matching Service geschrieben, um den nächsten Schritt im Review-Prozess einzuleiten.

Im Rahmen des MVP wurde der Configuration Service mit insgesamt acht Plugins ausgestattet. Drei Plugins definieren die Review-Typen (Einfachblind, Doppelblind und Offen) und fünf weitere Plugins spezifizieren die Review-Formate (Einzelarbeit, Gruppenarbeit, Bachelor Thesis, Master Thesis und Seminararbeit). Die Erstellung neuer Plugins ist einfach und erfordert lediglich die Implementierung eines Plugin Interfaces, woraufhin die daraus resultierende Archivdatei im Format JAR in das Plugin-Verzeichnis des Configuration Service kopiert oder eingebunden werden muss.

3.4 Server-Sent Events (SSE) im Communication und Notification Service

Eine architektonische Besonderheit des Communication und Notification Services ist die Implementierung von SSE. SSE etabliert eine unidirektionale Verbindung vom Server zum Client, über die der Server asynchron Live-Updates und Benachrichtigungen

pushen kann. Im Vergleich zu WebSockets, die bidirektional und in der Handhabung deutlich komplexer sind, stellt SSE für diesen Anwendungsfall eine leichtgewichtigere Alternative dar. Auch gegenüber clientseitigem Polling (z. B. Short oder Long Polling) bietet SSE den Vorteil, dass Updates in Echtzeit und mit stark reduziertem Overhead (ohne ständigen Verbindungsaufbau) übertragen werden.

Der Einsatz von SSE hat sich für die Chat- und Benachrichtigungsfunktionen als äußerst zuverlässig erwiesen. Es existiert jedoch eine signifikante infrastrukturelle Einschränkung durch das API Gateway von AWS, welches ein hartes Timeout von 29 Sekunden für aktive Verbindungen erzwingt. Für eine langlebige Verbindung über SSE ist dies im Standardbetrieb nicht praktikabel.

Um dieses Limit zu umgehen, beendet der Server die Verbindung über SSE proaktiv nach 25 Sekunden. Da das Protokoll SSE nativ über Selbstheilungsmechanismen (Auto-Reconnect) verfügt, erkennt der Client den Verbindungsabbruch sofort und baut die Verbindung automatisch neu auf. Dadurch wird ein scheinbar unterbrechungsfreier Live-Datenstrom simuliert.

3.5 Strategy Pattern im Notification Service

Der Notification Service verwendet das Strategy Pattern, um die verschiedenen Versandkanäle von der eigentlichen Zustellung zu entkoppeln. Jeder Kanal implementiert das gemeinsame Interface `NotificationChannel`; der `NotificationDispatcher` wählt anhand des angeforderten Kanaltyps die passende Strategie aus und führt deren Versand aus. Aktuell existieren Strategien für In-App-Benachrichtigungen, E-Mail, Slack und Discord. Dadurch bleiben die aufrufenden Services von kanalspezifischen Details wie SSE, SMTP oder Webhooks unabhängig, während Versandversuche einheitlich protokolliert werden. Im MVP können Nutzer über die Web-UI jedoch noch keine zusätzlichen Empfänger oder externe Kanäle individuell konfigurieren. Künftig sollten solche Präferenzen über die Benutzeroberfläche verwaltet und beim Erzeugen der Benachrichtigung berücksichtigt werden. Weitere Versandwege lassen sich dann durch eine zusätzliche Strategie ergänzen, ohne den Dispatcher anzupassen.

3.6 Web-UI

Der Service der Web-UI ist für die Bereitstellung der Weboberfläche des PeerReview-Systems verantwortlich. Sie nimmt in der komponenten- und ereignisgesteuerten Gesamtanwendung eine zentrale Rolle ein, da sie zur Ermöglichung von Benutzerinteraktionen Daten bereitstellen muss, welche auf diverse Services verteilt vorliegen. Hierdurch wird einerseits die Skalierung von einzelnen Services ermöglicht, andererseits wird jedoch gleichzeitig eine Datenaggregation erforderlich, bevor die Weboberfläche

vollständig angezeigt werden kann. Angesichts des Umfangs des Projekts wurde sich in diesem Zusammenhang dazu entschieden, kein dediziertes Backend for Frontend (BFF) zu implementieren, welches diese Aufgabe übernehmen könnte. Folglich obliegt es der Web-UI, die notwendigen Daten von den verschiedenen Services abzurufen und zusammenzuführen. Dies soll dazu beitragen, die Weboberfläche weiter zu entkoppeln und infrastrukturelle Komplexität gemäß der Idee zur Implementierung eines MVP zu reduzieren. Die Weboberfläche ist zustandslos und verwendet hiervon abweichend lediglich den LocalStorage des Browsers, um Einstellungen wie den präferierten Ansichtsmodus des Benutzers zu speichern. Konzeptionell erklärt sich aus diesen Bedingungen, weshalb die Weboberfläche als Single Page Application (SPA) unter Verwendung von React und Vite implementiert wurde. Dies ermöglicht es, Daten von verschiedenen Services asynchron abzurufen und die Benutzeroberfläche dynamisch und schrittweise zu aktualisieren, ohne dass eine vollständige Seitenaktualisierung erforderlich wird. Die zusätzliche Verwendung eines CDN erlaubt es zudem, die Latenzzeit der Weboberfläche zu reduzieren.

Zur Kommunikation mit den Backend-Services werden von der Web-UI ausschließlich Clients verwendet, welche anhand der Spezifikationen gemäß OpenAPI der jeweiligen Services automatisiert generiert wurden. Der Aufbau des Projekts als Mono-Repository trägt hierbei dazu bei, dass die Schnittstellen leicht überprüfbar bleiben und inkompatible Änderungen frühzeitig bemerkt werden. Anpassungen an einem Service erfordern lediglich die Neugenerierung des jeweiligen Clients. In diesem Zuge wurde auf die explizite Einführung von Data-Access-Objekten innerhalb der Weboberfläche verzichtet. Sollte die Komplexität des Projekts durch zukünftige Erweiterungen steigen oder externe Services angebunden werden, sollte dies erneut evaluiert werden.

Die Weboberfläche ist aufgrund der Verwendung von AWS Cognito nicht für die Bereitstellung einer Anmeldeseite zuständig. Stattdessen wird die Authentifizierung durch eine Umleitung auf eine von AWS angebotene Anmeldeseite durchgeführt. Nach erfolgreicher Authentifizierung wird der Benutzer zurück auf die Weboberfläche geleitet, wobei ein Session-Token übergeben wird, welches für die Dauer der Sitzung gültig ist. Dieses Token wird von der Weboberfläche bei jedem Request an die Backend-Services übermittelt, um die Identität des Benutzers zu verifizieren und den Zugriff auf die jeweiligen Ressourcen zu autorisieren. Gleiches gilt für den Logout, bei welchem der Benutzer kurzzeitig ebenfalls umgeleitet wird, um die Sitzung zu beenden.

3.7 DevOps und Infrastructure as Code

Zu einer modernen Softwarearchitektur gehören neben dem reinen Applikations- und Infrastrukturdesign zwingend auch DevOps-Praktiken, die ein kontinuierliches Testen, Bauen und Ausrollen durch CI und CD ermöglichen. Die Architektur dieser Pipeline

ist modular aufgebaut, wodurch sich neue Services aufwandsarm integrieren lassen. Der genaue Entwicklungszyklus wird in Abschnitt 4.3 detailliert beschrieben.

Um das Deployment in die Cloud von AWS vollautomatisiert durchzuführen, wurde IaC angewandt. Die Infrastruktur wird dabei in imperativen Code geschrieben, welcher innerhalb der Pipeline ausgeführt wird, um die entsprechenden Cloud-Ressourcen zu provisionieren oder zu aktualisieren. Zum Einsatz kam hierbei CDK in der Programmiersprache TypeScript.

Auf Code-Ebene nutzen die Komponenten für IaC eine eigens entwickelte InfraLibrary. Diese abstrahiert und standardisiert komplexe Infrastruktur-Setups, wie das Deployment eines Service auf ECS inklusive Lambda-Proxy, Routen des API Gateways und Integration von AVP, in wenigen, wiederverwendbaren Zeilen Code. Durch diese Abstraktion wird sichergestellt, dass alle Services auf identischen infrastrukturellen Mustern basieren. Dies gewährleistet eine hohe Konsistenz, erleichtert das Debugging enorm und ermöglicht es, infrastrukturelle Anpassungen zentral vorzunehmen und global auf alle Services auszurollen.

Ergänzend zur InfraLibrary existiert das Projekt InfraBaseline. Dieses provisioniert die grundlegende Netzwerkinfrastruktur, den Cognito-Userpool auf AWS, die Cloud-Map-Namespaces sowie die CloudFront-Distribution.

Der massive strategische Vorteil von IaC zeigte sich im Projektverlauf mehrfach. So konnten Features isoliert getestet und die Infrastruktur jederzeit deterministisch auf ältere Stände zurückgerollt werden, wobei Applikationscode und Infrastruktur stets konsistent blieben. Besonders erfolgskritisch war der Ansatz mit IaC, als der ursprüngliche Account bei AWS aufgrund eines abgelaufenen Leases gesperrt wurde. Dank der vollständigen Implementierung als Code konnten sämtliche Ressourcen nahezu ohne manuelle Eingriffe in einem neuen Account hochgefahren werden. Ohne IaC hätte dieser Vorfall eine stundenlange, extrem fehleranfällige manuelle Neuprovisionierung erfordert.

4 Umsetzung und Workflow

Ein wesentlicher Bestandteil der praktischen Umsetzung umfasste den Einsatz von Coding-Agenten auf Basis von KI wie Claude Code oder Google Antigravity. Vor Beginn der eigentlichen Projektphase wurden strategische Entscheidungen bezüglich deren Evaluierung getroffen, um eine bestmögliche Integration in den Entwicklungsprozess zu gewährleisten. Neben diesen modernen, Large Language Model (LLM)-basierten Ansätzen fanden klassische Verfahren des Software-Lebenszyklus Anwendung. Hierzu zählen die Einrichtung einer Pipeline für CI und CD sowie das automatisierte und manuelle Testen.

4.1 Baseline

Zu Beginn der Implementierungsphase wurde die Entscheidung für ein Mono-Repository (Monorepo) getroffen. Die primäre Absicht bestand darin, den KI-Agenten den vollständigen Kontext über alle Microservices hinweg agnostisch bereitzustellen, ohne dass ein isolierter Wechsel zwischen verschiedenen Projektarchiven erforderlich ist. Obwohl es sich bei dem Vorhaben um eine vollständige Greenfield-Entwicklung handelte, wurde die initiale Baseline bewusst nicht agentisch, sondern lediglich KI-unterstützt und manuell entwickelt.

Im Rahmen dieser Baseline wurden alle grundlegenden Infrastrukturkomponenten mittels CDK implementiert. Dadurch konnte sichergestellt werden, dass die Services keine kostenintensiven Default-Ressourcen verwenden. Gleichzeitig förderte dieses Vorgehen unser Systemverständnis, was das spätere Debugging erheblich erleichterte. Da die Inter-Service-Kommunikation vollständig transparent war, ließ sich die Ursachenanalyse (Root-Cause-Analysis) im Fehlerfall in der Regel auf einzelne, isolierte Microservices eingrenzen. Dies erwies sich in einem verteilten System als fundamentaler Vorteil: Da autonome Agenten in komplexen Cloud-Infrastrukturen auf AWS ohne vordefinierten Rahmen den Gesamtüberblick verlieren können, war es hocheffizient, den betroffenen Service vorab manuell zu identifizieren und den Agenten anschließend mit dedizierten Fehlermeldungen in einer isolierten Umgebung arbeiten zu lassen.

Ergänzend zur infrastrukturellen Basis wurde eine applikative Grundlage in Form eines Template Service geschaffen. Dieser diente den Coding-Agenten als struktureller Anker und Referenzarchitektur, um nahezu direkt lauffähige, neue Services zu generieren. Zu diesem Zweck wurde dieser Prototyp manuell implementiert, mit minimalen funktionalen Features ausgestattet und so auf die Infrastruktur abgestimmt, dass das Zusammenspiel aus Netzwerk, Autoscaling, DynamoDB-Datenbanken und dem API Gateway fehlerfrei funktionierte. Erst nach erfolgreicher Validierung dieser Basis wurden

die Agenten eingesetzt, um die eigentliche Geschäftslogik in Kopien dieses Template-Service zu implementieren.

Die Praxis zeigte, dass die Coding-Agenten die vorgegebene Baseline sowohl auf infrastruktureller als auch auf applikativer Ebene weitgehend adaptierten, was zu einer konsistenten Systemlandschaft führte. Dennoch war auffällig, dass die Agenten vereinzelt unvorhergesehene und nicht an die Architekturrichtlinien angepasste Entscheidungen trafen. Beispielsweise versuchte ein Agent eigenständig, eine PostgreSQL-Datenbank zu provisionieren, was für den vorliegenden Anwendungsfall fachlich nicht erforderlich war und zudem durch den Verzicht auf vollständige Serverless-Eigenschaften die Betriebskosten unnötig erhöht hätte.

Auch der Einsatz des Monorepos wirkte sich im Hinblick auf das Kontextfenster (Context Window) der verwendeten Sprachmodelle positiv aus, da die Modelle jederzeit serviceübergreifendes Systemwissen in ihre Generierung einbeziehen konnten. Aus ökonomischer Sicht offenbarte dieser Ansatz jedoch auch Nachteile: Bei der Initiierung neuer Prompts fiel das Kontextfeld und insbesondere die Anzahl der Input-Tokens sehr groß aus, was zu erhöhten Gebühren für APIs führte. Zudem besteht bei sehr großen Kontexten das inhärente Risiko von Qualitätseinbußen, da LLM dazu neigen, feine Details inmitten umfangreicher Datenmengen zu übersehen (Lost-in-the-Middle-Phänomen).

4.2 Agentic Coding

Unter Agentic Coding versteht man den Einsatz autonomer Software-Agenten auf Basis von LLMs, die über klassische Code-Vervollständigung hinausgehen. Im Gegensatz zu Assistenzsystemen versucht man mit Coding-Agenten einen kontinuierlichen Zyklus (Agentic Loop) aus Analyse, Planung, Umsetzung sowie anschließender Verifikation abzubilden. Hierfür werden orchestrierende Agentensysteme häufig in Form von CLI-Anwendungen oder IDE-Integrationen verwendet, die den Agenten eine strukturierte Ausführungsumgebung bieten. Dies soll die selbstständige Bearbeitung komplexer Entwicklungsaufgaben ermöglichen. Im Rahmen dieser Arbeit wurden verschiedene solcher agentischen Werkzeuge evaluiert, um deren Eignung für den Aufbau einer ereignisgesteuerten Microservice-Architektur festzustellen.

4.2.1 Agents.MD / Claude.MD

Ein wichtiger Aspekt guter Softwareentwicklung besteht darin, dass Architekturentscheidungen und Konventionen allen an der Software arbeitenden Entwicklern bekannt sein müssen, um diese anwenden zu können. Dieses Selbstverständnis führt zwangsweise dazu, dass auch autonom agierende Agenten die existierenden Regeln beachten müssen. Nur so kann Konsistenz gewährleistet werden. Um Coding-Agenten ebensolche

projektbezogene Instruktionen und Kontext bereitzustellen, hat sich zunehmend die plattformübergreifende `AGENTS.md` als offener Standard [1] etabliert, der von Editoren wie Cursor und Agentensystemen wie Codex gelesen und respektiert wird. Im Wesentlichen handelt es sich hierbei um eine Markdown-Datei, die ähnliche Funktionen wie die klassische `README.md` erfüllt, jedoch speziell für die Bereitstellung von Richtlinien, Konventionen und Projektinformationen für KI-Agenten konzipiert ist. Üblicherweise wird diese Datei im Wurzelverzeichnis eines Projekts abgelegt, um den Agenten einen zentralen Einstiegspunkt zu bieten. Oftmals ist es jedoch auch möglich, mehrere `AGENTS.md`-Dateien zu verwenden und diese in Unterverzeichnissen zu platzieren, um den Agenten kontextspezifische Informationen zu liefern. Hierzu sei angemerkt, dass durch die `AGENTS.md` zwar ein verbreiteter Standard existiert, es jedoch keine Garantie gibt, dass dieser stets eingehalten wird. So gibt es bis heute KI-Werkzeuge, die den Standard nicht respektieren. Dies führt zum Beispiel dazu, dass der Hersteller Anthropic mit Claude Code weiterhin ausschließlich sein proprietäres Dateiformat `CLAUDE.md` beachtet [2]. Die Idee ist ansonsten identisch. Da im Rahmen dieser Arbeit mit verschiedenen Agentensystemen gearbeitet werden sollte, war es aus diesem Grund erforderlich, ebendiesen Umstand zu beachten und zu adressieren. So wurde neben der `AGENTS.md` auch eine `CLAUDE.md` erstellt, wobei letztere mithilfe eines `@Agents.md`-Ausdrucks auf die `AGENTS.md` verweist.

Während des Projektverlaufs konnte immer wieder festgestellt werden, dass Agenten die `AGENTS.md`-Dateien erfolgreich gelesen hatten und deren Anweisungen bestmöglich befolgten. Zeitgleich wurde in Bezug auf die Dateien allerdings auch schnell ein Problem deutlich: Vorgaben können veralten oder im Konflikt stehen. Bei Änderungen an Architekturmustern ist es wiederholt dazu gekommen, dass `AGENTS.md`-Dateien nicht angepasst wurden, was zu späteren Zeitpunkten zu Problemen geführt hat. Hierauf muss geachtet werden.

4.2.2 Antigravity

Google's Antigravity [3] wird in drei grundlegend unterschiedlichen Versionen angeboten. Antigravity IDE [4] ergänzt die gewohnte Entwicklungsumgebung VS Code [5] mit zahlreichen Werkzeugen der KI.

Antigravity 2.0 [6] ist eine Agentenorchestrierungsplattform, die den Texteditor entfernt und die Möglichkeit liefert, auf verschiedene Art und Weise mit Agenten der KI zu interagieren. Mit Antigravity 2.0 lassen sich außerdem Routineaufgaben planen und automatisiert erledigen. So kann zum Beispiel stündlich verlangt werden, mögliche Pull Requests zu mergen. Sogenannte Skills liefern den Agenten verschiedene Beschreibungen wie bestimmte Aufgaben zu erledigen sind.

Zuletzt wird Antigravity, ähnlich wie Claude Code, auch als eine Kommandozeilenanwendung bereitgestellt [7]. Diese bietet dem Nutzer ein ähnliches Erlebnis wie in Antigravity 2.0, beschränkt sich aber auf die Nutzung per Kommandozeile. Diese KI-Werkzeuge bieten den Sprachmodellen nicht nur Zugriff auf verschiedenste Kommandozeilenanwendungen und somit Zugriff auf bestimmte Bereiche der Hostmaschine, sondern ermöglichen es den Modellen ebenso weitere Subagenten zu erzeugen. Diese Subagenten werden erzeugt, um die für die Aufgabe als notwendig erachteten Teilaufgaben zu erledigen.

Für dieses Projekt haben wir Antigravity 2.0 genutzt, um den von Konzernen gewünschten State-of-the-Art zu erleben und erlernen. Für jeden Microservice konnte ein neuer Kontext erstellt werden. Bevor der Agent zu implementieren beginnt, wird ein Planungsmodus verwendet, um den notwendigen Kontext einzulesen und sicherzustellen, dass die Implementierung Schritt für Schritt erfolgt. Hierzu stellt der Agent teilweise beeindruckende Grafiken zur Architektur bereit. Der Plan lässt sich außerdem iterativ überarbeiten. Wenn der Plan bestätigt wird, implementiert die KI alle notwendigen Teile, so zumindest die Idee.

4.2.3 Claude Code

Claude Code [8] ist die Agentenplattform des Herstellers Anthropic und wurde im Rahmen dieses Projekts am intensivsten eingesetzt. Neben der Kommandozeilenanwendung steht das Werkzeug auch als Extension für gängige IDEs wie VS Code zur Verfügung, was die im Rahmen dieser Arbeit primär genutzte Variante war; der Agent liest und editiert Dateien eigenständig, führt Shell-Befehle (u. a. `mvn`, `npm`, `git`, `cdk`) aus und lässt sich, ähnlich wie bei Antigravity 2.0, über einen eigenen Planungsmodus (Plan Mode) zunächst einen Umsetzungsplan erarbeiten und bestätigen, bevor die eigentliche Implementierung beginnt. Wie auch die übrigen betrachteten Werkzeuge kann Claude Code bei Bedarf Subagenten für klar abgegrenzte Teilaufgaben erzeugen. Die projektspezifischen Konventionen wurden dem Werkzeug, wie in Abschnitt 4.2.1 beschrieben, über die Datei `CLAUDE.md` bereitgestellt.

Im direkten Vergleich zu den übrigen im Projekt eingesetzten Werkzeugen lieferte Claude Code durch den Zugriff auf sehr leistungsfähige Modelle (u. a. Claude Opus 4.8) tendenziell die qualitativ überzeugendsten Ergebnisse, war dabei aber mit Abstand das teuerste Werkzeug. Die dadurch entstehenden hohen Token-Kosten führten in der Praxis dazu, dass bewusst sparsamer promptet und iteriert wurde als mit günstigeren Alternativen, was die eigentlich zu erwartende Entwicklungsgeschwindigkeit spürbar relativierte.

Am Beispiel der Implementierung von Notification Service und Response Service, die beide vollständig mit Claude Code entwickelt wurden, zeigte sich, dass das Werkzeug

bei klar vorgegebenem architektonischem Kontext solide Ergebnisse liefert: Etablierte Muster aus anderen Services des Monorepos, etwa das Single-Table-Design von DynamoDB oder die Verwendung vorsignierter URLs für den Zugriff auf S3, wurden korrekt übernommen und konsistent weitergeführt, ohne dass diese im Detail vorgegeben werden mussten. Auch eigenständig vorgeschlagene Entwurfsmuster, etwa ein Strategy Pattern zur Anbindung mehrerer Benachrichtigungs Kanäle (In-App, Discord, E-Mail, Slack), erwiesen sich als sinnvoll und leicht um weitere Kanäle erweiterbar.

Gleichzeitig zeigte sich wiederholt, dass sich die Ausgaben des Werkzeugs nicht ungeprüft übernehmen lassen. So erzeugte Claude Code beispielsweise eine Beschreibung des Notification-Controllers, die dessen tatsächliches Verhalten falsch wiedergab: Die Dokumentation behauptete, es werde stets ein einheitlicher Erfolgsstatus zurückgegeben, obwohl der Code bereits differenziert zwischen vollständigem, teilweisem und fehlgeschlagenem Versand einer Benachrichtigung unterschied. Dieser Widerspruch fiel erst bei einer nachträglichen Prüfung auf und musste korrigiert werden. Ähnlich gelagert waren Fehler, die lokal unentdeckt blieben, weil einzelne Komponenten wie ein Listener für SQS lokal per Konfiguration deaktiviert sind, und erst beim tatsächlichen Deployment sichtbar geworden wären, etwa ein zunächst fehlender, explizit zu definierender `ObjectMapper`-Bean. Auch typische Fallstricke aus dem Zusammenspiel von Lombok und dem SDK von AWS (z. B. methodenbasierte DynamoDB-Annotationen, die nicht auf Lombok-generierten Feldern stehen dürfen) traten zunächst unbemerkt auf und mussten identifiziert werden. Diese Erfahrungen bestätigen den in Abschnitt 4.2.1 beschriebenen Befund: Eine `CLAUDE.md`-Datei ist kein einmalig geschriebenes, vollständiges Regelwerk, sondern wächst erst durch das Auffangen konkreter, selbst erlebter Fehler zu einem wirksamen Werkzeug heran. Ohne einen Entwickler, der die Ergebnisse aufmerksam prüft, gegen die tatsächliche Codebasis abgleicht und Vorgaben entsprechend nachschärft, wären mehrere dieser Probleme unbemerkt geblieben oder erst in Produktion aufgefallen.

4.2.4 OpenCode

Frameworks der KI wie Claude Code, Antigravity oder Codex haben gemeinsam, dass sie üblicherweise für die Softwareentwicklung in Kombination mit den hauseigenen LLMs und Abonnements der jeweiligen Hersteller wie Anthropic, Google oder OpenAI vorgesehen werden. Eine Anbindung der Systeme an LLMs von Drittherstellern oder selbstbetriebenen Instanzen wird nicht unterstützt oder obliegt vorbehaltlich zukünftigen Einschränkungen. Es wird aktiv versucht, die Verwendung der Abonnements außerhalb der jeweiligen Plattformen zu unterbinden und die Ökosysteme durch die Bindung an die proprietären Dienste zu stärken. Insbesondere Anthropic hat sich zum Beispiel bestrebt gezeigt, eine Verwendung mit nicht unternehmenseigener Software zu verhindern und den Zugriff von zuwiderhandelnden Benutzern vollständig zu sperren

[9]. Eine Migration von einem Anbieter zu einem anderen wird hierdurch erheblich erschwert, obwohl die zur Anbindung von LLMs verwendeten Schnittstellen üblicherweise einheitlich sind.

Bei dem Open-Source-Projekt OpenCode handelt es sich primär um eine CLI, welche eine Alternative zu den erwähnten proprietären Agentensystemen darstellen soll. Eine Variante mit GUI ist ebenfalls verfügbar, derzeit jedoch noch in einer frühen Entwicklungsphase. Es bietet die Möglichkeit, beliebige LLMs von verschiedensten Anbietern zu integrieren oder die Agenten auf selbstgehosteten Instanzen zu betreiben. OpenCode ist hochgradig konfigurierbar und erlaubt die Erweiterung durch Plugins. So ist es beispielsweise möglich, dass Agenten der KI, die auf verschiedenen Plattformen ausgeführt werden, miteinander kommunizieren und von einem einzigen kombinierten System orchestriert werden. Ein Austausch von verwendeten LLMs oder Anbietern gestaltet sich durch die offene Architektur denkbar einfach und kann üblicherweise ohne Modifikationen der Agenten oder sonstigen Konfigurationen erfolgen.

Angesichts der beschriebenen vermeintlichen Vorteile von OpenCode wurde sich dazu entschieden, auch dieses Werkzeug im Rahmen der Entwicklungsphase dieser Arbeit zu betrachten. So wurden zunächst sowohl diverse GPT- als auch Gemini-Modelle von OpenAI bzw. Google durch die Schnittstelle zur Anwendungsprogrammierung von Microsoft Azure und andere Abonnements an OpenCode angebunden, um diese für verschiedene Agenten zu verwenden. Es wurde zum Beispiel ein Architect-Agent definiert, welcher das Modell GPT-5.4 verwendet und für die Planung und Orchestrierung von Featureimplementierungen zuständig ist. Die Agenten, die an der tatsächlichen Implementierung arbeiten, wurden hingegen auf Basis von GPT-5.3-Codex ausgeführt, die ihr Verständnis vom Projekt wiederum von Explorer-Agenten mit Gemini-2.5-Flash beziehen. Auf diese Weise konnte einzelnen Agenten ein spezifisches Rollenverständnis zugewiesen werden, wodurch versucht wurde, die Stärken der jeweiligen Modelle gezielt zu nutzen. Die genaue verwendete Konfiguration kann im Ordner `.opencode` im Projektverzeichnis eingesehen werden.

Insbesondere der Service der Web-UI wurde maßgeblich mithilfe der eben beschriebenen Konstellation von Agenten und Modellen erfolgreich implementiert. Ein signifikanter Unterschied zwischen OpenCode und den proprietären Agentensystemen konnte dabei nicht festgestellt werden. Es gilt jedoch festzuhalten, dass zur effektiven Verwendung von OpenCode ein gewisses Maß an Einarbeitung empfehlenswert ist. Insbesondere die Konfiguration von Agenten mit Rollen trug stark zur Qualität der Ergebnisse bei, wobei entstehende Kosten für APIs durch die Auswahl von zur Komplexität passenden LLM-Modellen erheblich reduziert werden konnten. Eine derartige manuelle Konfiguration erschien bei den proprietären Agentensystemen weniger erforderlich, führte jedoch auch dazu, dass die Systeme weniger an den individuellen Anwendungsfall angepasst wurden.

4.2.5 GitHub Copilot Reviewer

Commit und Pull Requests von agentischer Entwicklung umfassen häufig große Mengen an Code - vor allem in Greenfield-Situationen wie in diesem Projekt. Um den Überblick über diese Änderungen behalten zu können, liefert GitHub mit GitHub Copilot Reviewer eine in die Plattform integrierte Unterstützung durch KI [10]. Dieses Produkt soll dabei helfen, die Quellcodeänderungen zu verstehen, Fehler zu finden oder auch selbstdefinierte Anweisungen durchzuführen. Die Review durch KI kann in den Arbeitsfluss für CI und CD eingebunden werden, was einen hohen Grad an Automatisierung erlaubt.

Unsere Erfahrungen mit Copilot Review belaufen sich auf wenige Tests. In einem Drittel ($\frac{1}{3}$) der Fälle wurde eine falsche Problemlösung vorgeschlagen.

4.3 Integration von Continuous Integration (CI) und Continuous Deployment (CD)

Als etabliertes Verfahren der modernen Softwareentwicklung wurde eine Pipeline für CI und CD eingerichtet. Diese erlaubt ein automatisiertes Deployment der aktuellen Softwareversion auf Knopfdruck. Hierdurch werden fehleranfällige manuelle Bereitstellungsprozesse eliminiert. Gleichzeitig stellt der Ansatz der kontinuierlichen Integration sicher, dass Modifikationen frühzeitig zusammengeführt werden, wodurch potenzielle Integrationskonflikte oder Laufzeitfehler in einem frühen Stadium sichtbar werden. Neben der Option, spezifische Feature-Branches manuell zu deployen, wird bei jedem Merge auf den geschützten Hauptzweig (`main branch`) automatisch das Deployment aller vom Commit betroffenen Microservices initiiert.

Die Pipeline ist technologisch auf Basis von GitHub Actions realisiert. Der Workflow umfasst das automatisierte Kompilieren des Java-Backend-Codes sowie die Ausführung der Unit-Tests für die Applikation und den TypeScript-basierten Code für IaC. Nach erfolgreicher Verifikation wird das Docker-Image erstellt und in ECR gepusht. Im anschließenden Schritt initiiert die Pipeline mittels des Befehls `cdk deploy` das infrastrukturelle Deployment, wodurch sowohl die modifizierten Cloud-Ressourcen aktualisiert als auch die neuen Applikations-Images auf Instanzen mit ECS Fargate ausgerollt werden.

Lokal lässt sich die Software nicht ausführen, weil eine hohe Abhängigkeit zu Cloud-Ressourcen besteht. Auch wenn einige Services, wie eine asynchrone Queue, lokal abgebildet werden könnten, ist die Integration von Cognito, dem API Gateway, DynamoDB oder Verified Permissions auf AWS sehr komplex und aus diesem Grund nicht Teil des MVP.

Die Web-UI bietet jedoch die Möglichkeit, lokal gestartet zu werden und weiterhin die Backend-Services zu nutzen. Dazu kommt ein Reverse Proxy zum Einsatz, der alle Anfragen der lokal instanziierten Web-UI an die Umgebung auf AWS weiterleitet. Hierzu wird der Befehl `npm run dev` im Verzeichnis der Web-UI ausgeführt. Einige Features, wie beispielsweise Ereignisse über SSE, sind hierbei zwar limitiert, was für eine Entwicklungsumgebung aber akzeptabel ist.

4.4 Manuelles Testen

Neben der automatisierten Testabdeckung wurde das System während der gesamten Entwicklungszeit auch manuell überprüft. Im Vordergrund stand dabei ein klassischer Klicktest über die Web-UI, bei dem die zentralen Abläufe wie Einreichung, Zuweisung und Begutachtung wiederholt durchgespielt wurden. Ergänzend wurden einzelne REST-Endpunkte gezielt über Postman angesprochen, um Anfragen unabhängig von der Oberfläche zu prüfen. Da ein wesentlicher Teil der Kommunikation zwischen den Services über SQS erfolgt, wurden zudem Nachrichten manuell in die jeweiligen Queues geschrieben und die daraus resultierende Verarbeitung sowie die abgelegten Ergebnisse kontrolliert.

Auffälligkeiten aus diesen Tests wurden nicht separat dokumentiert, sondern direkt im Code behoben und über die bestehende CI/CD Pipeline (siehe Abschnitt 4.3) umgehend wieder ausgerollt. Dieses pragmatische Vorgehen war durch die geringe Teamgröße und die kurzen Deployment-Zyklen möglich und sorgte für eine schnelle Rückkopplung zwischen Testen und Implementierung.

4.5 Automatisierte Testabdeckung

Neben der manuellen Qualitätssicherung wurde großer Wert auf eine fundierte automatisierte Testabdeckung gelegt. Da das System eine verteilte Microservice-Architektur aufweist, lag der Fokus auf der Isolation einzelner Domänen mittels Unit-Tests. Für die Backend-Services (Java/Spring Boot) wurde das Mockito-Framework eingesetzt, um externe Abhängigkeiten wie Datenbanken oder Dienste von AWS (z. B. S3 und SQS) zuverlässig zu simulieren. Die Metriken zur Codeabdeckung wurden durch das JaCoCo-Plugin ermittelt und direkt als Qualitätsgate in den Build-Prozess (`mvn verify`) integriert.

Im Frontend (`web-ui`) erfolgte die automatisierte Prüfung mittels Vitest und der React Testing Library. Hierbei wurden sowohl funktionale Logik als auch komplexe Interaktionen der UI isoliert getestet. Auch im Frontend wurde die Metrikerfassung in den Workflow für CI eingebunden.

Im Infrastrukturbereich wurde durch das Anwenden von CDK die Möglichkeit geschaffen, auch den Infrastrukturcode automatisch durch Unit-Tests zu testen. Während der Entwicklung hat sich dabei gezeigt, dass sich dies insbesondere unter dem Einsatz von KI für die Entwicklung von Code als hilfreich erweist, da Tests mögliche ungewollte Infrastrukturänderungen aufdecken.

Wie im nachfolgenden Abschnitt näher erläutert wird (siehe Abschnitt 4.6), spielt die so ermittelte Testabdeckung eine tragende Rolle bei der qualitativen Bewertung des Quellcodes und bildete die mathematische Grundlage zur Berechnung des Index CRAP, durch welchen besonders fehleranfällige und unzureichend getestete Bereiche identifiziert werden konnten.

4.6 Optimierung

Während der Entwicklungsphase wurde mithilfe von ausführlichen Prompts für KI und entsprechend angepassten `AGENTS.md`-Dateien versucht, die Testabdeckung des Gesamtsystems sicherzustellen. Gleichzeitig wurde hierbei angestrebt, allgemeine Codestrukturen und Konventionen festzulegen. Schnell hat sich dies jedoch insbesondere durch die Verwendung von unterschiedlichen LLMs als kein triviales Unterfangen herausgestellt, da die Modelle jeweils mit ihren spezifischen Präferenzen und Verhaltensmustern daherkamen. So wurden beispielsweise Regeln unterschiedlich stark eingehalten, wodurch es schließlich dazu gekommen ist, dass die Testabdeckung für einige Services entweder stark oder vollkommen vernachlässigt wurde. Insbesondere der Service der Web-UI verblieb bis zu deren Fertigstellung ohne Tests. Dies verbesserte sich erst, nachdem eine Codeabdeckung explizit gefordert wurde. Es ist davon auszugehen, dass besser ausformulierte `AGENTS.md`-Dateien diese Situation hätten verbessern können. Zeitgleich hätte dies bedeutet, dass bestenfalls bereits zu Anfang des Projekts fertige Architekturpläne, Ordnerstrukturen und Konventionen hätten bereitstehen müssen. Dies war uns nicht möglich und erschien wenig praktikabel.

Dieser eben beschriebene erste negative Eindruck bezüglich der Testabdeckung wurde nach Fertigstellung der Implementierung des MVP weiter untersucht. Hierzu wurde zunächst die Codeabdeckung der einzelnen Microservices des PeerReview-Systems ermittelt, um anschließend den jeweiligen CRAP-Index zu berechnen. Für diesen wurde festgelegt, dass ein maximaler Wert von 30 zu akzeptieren ist. Beim Vergleich der Indexwerte mit dem Schwellwert wurden daraufhin diverse Codestellen identifiziert, die basierend auf den geltenden Bedingungen als kritisch einzustufen waren und einer Optimierung bedurften (siehe Tabelle 2).

Service	Klasse	Methode	CRAP-Score
Web-UI	Submissions.tsx	mapConfigToDisplay	407.1
Web-UI	Dashboard.tsx	fetchTasks	314.5
Web-UI	SubmissionModal.tsx	SubmissionModal	197.5
Communication-Service	ChatService	getChat	132.0
Response-Service	BedrockProxyClient	generateReview	132.0

Tabelle 2: Top-Ausreißer beim Index Change Risk Anti-Patterns (CRAP) vor Optimierungen

Um diese identifizierten Ausreißer zu beheben, wurden durch KI unterstützte Refactoring-Maßnahmen durchgeführt. Im Backend-Bereich (z. B. im Communication-Service und Response-Service) lag der Fokus primär auf der Erhöhung der Testabdeckung durch die automatisierte Generierung von Mockito-Unit-Tests. Da die unzureichende Codeabdeckung eine zentrale Rolle in der Formel für CRAP spielt, führte eine Erhöhung der Testabdeckung zu einer drastischen Reduktion des Indexwertes auf die zugrundeliegende zyklomatische Komplexität.

In der Web-UI wurden komplexe React-Komponenten hingegen primär strukturell zerlegt. Verschachtelte Codeblöcke und umfangreiche Logikzweige wurden in kleinere Hilfsfunktionen mit einer zyklomatischen Komplexität von maximal 5 extrahiert.

Das Ergebnis dieser Optimierungsmaßnahmen ist in Tabelle 3 dargestellt. Sämtliche zuvor kritischen Methoden konnten erfolgreich auf oder unter den festgelegten Schwellwert von 30 gesenkt werden.

Service	Klasse	Methode	CRAP-Score
User-Service	CognitoService	searchUsers	30.0
Communication-Service	ChatRepository	findMessages	30.0
Response-Service	ResultService	isAuthorOfSubmission	30.0
Notification-Service	NotificationSseService	push	20.0
Web-UI / Backend	(Alle verbleibenden)	(Diverse)	< 20.0

Tabelle 3: Höchste verbleibende Werte für den Index Change Risk Anti-Patterns (CRAP) nach den Optimierungen

4.7 Dokumentation

Für die Endpunkte der API liefern Spezifikationen gemäß OpenAPI [11] umfangreiche Dokumentation. Aus diesen Spezifikationen wird durch ein Python Skript eine Kollektion an Postman [12] Ressourcen erstellt. Neben den eigentlichen Requests werden dabei auch Environment-Dateien generiert, welche die für die Cognito-Authentifizierung notwendigen Variablen bereitstellen. Zugangsdaten selbst werden nicht versioniert, sondern zur Laufzeit über den Postman Vault referenziert. Ergänzend existieren, analog zu der in Abschnitt 4.2.1 beschriebenen **AGENTS.md**, weitere **AGENTS.md**-Dateien in einzelnen Unterverzeichnissen (z. B. im **postman**-Ordner), die den Agenten spezifische Architekturentscheidungen, Endpunkte der API, Datenbankenschemata und Beispiel-Payloads bereitstellen. Für die Visualisierung der Architektur wurde die Diagrammsoftware Draw.io [13] verwendet. Typst [14], ein modernes, in Berlin entwickeltes Textsatzsystem, wurde genutzt, um den Projektbericht zu schreiben und in eine Datei im Format PDF zu übersetzen. Neben der zentralen **README.md** im Wurzelverzeichnis des Repositories, welche einen Überblick über das Gesamtprojekt bietet, hat die KI die einzelnen Microservices nach dem Generieren des Codes zusätzlich jeweils in einer gesonderten **README.md** im Wurzelverzeichnis des jeweiligen Microservices dokumentiert. Da diese Dokumentationsartefakte überwiegend einmalig generiert werden, besteht analog zu der bereits in Abschnitt 4.2.1 beschriebenen Problematik das Risiko, dass sie bei späteren Codeänderungen nicht konsequent nachgepflegt werden und veralten.

5 Kritik und Einordnung des Vibecoding

Unter dem Begriff des „Vibe Coding“ beschreibt Andrej Karpathy im Februar 2025 eine neue Entwicklungsphilosophie, bei der das manuelle Schreiben von Code nahezu vollständig durch die Interaktion mit agentischen, auf großen Sprachmodellen basierenden Werkzeugen wie Antigravity oder Claude Code ersetzt wird [15].

5.1 Was lief gut

Ein Mono-Repo ermöglichte der KI zumindest grundsätzlich ein umfassenderes Kontextwissen über das Gesamtsystem, wenngleich fraglich bleibt, ob dieses Potenzial in der Praxis auch immer ausgeschöpft wurde. Leider führte dieser Ansatz durch den höheren Tokenverbrauch zu spürbar höheren Kosten (36 Euro für 2 Prompts). Positiv fiel vor allem die enorme Entwicklungsgeschwindigkeit auf. Insbesondere die Umsetzung des Frontends verlief gut. Dieser Geschwindigkeitsvorteil erwies sich, wie im folgenden Abschnitt beschrieben, zugleich als frustrierend. Nahezu alle im Projekt aufgetretenen Fehler konnten zudem mehr oder weniger zuverlässig von der KI behoben werden, und selbst ohne jegliche Vorerfahrung mit dem eingesetzten Tech-Stack (AWS, Spring Boot) war es möglich, eigenständig Features zu entwickeln und auszuliefern. Für klassische, in sich abgeschlossene Fragestellungen erwies sich der Einsatz generativer KI als sehr hilfreich.

5.2 Was lief eher schlecht

Den beschriebenen Vorteilen stand eine Reihe gravierender Probleme gegenüber. Es fehlt hier häufig jede Kontrolle darüber, was am Ende tatsächlich entsteht und wie dieses Ergebnis zustande kommt, während gleichzeitig eine unfassbar hohe Abhängigkeit (*Vendor Lock-in*) von den Anbietern entsteht. In der Gesamteinordnung eignet sich dieser Ansatz daher vor allem für einen Proof of Concept, sobald jedoch komplexere Software umgesetzt werden muss, stößt er an seine Grenzen. Jede Zeile Code, die der Agent (oder ein Mensch) schreibt, muss gewartet werden. Besonders bei komplexer Software ist die Unmenge an Code, die ein KI-Werkzeug erzeugt, nicht ideal. Besonders frustrierend war, dass letztlich niemand im Team mehr genau nachvollziehen konnte, wie die Anwendung im Detail funktioniert. Man promptet einen Fehler in die KI, wartet und hofft lediglich, dass die vorgeschlagene Lösung passt, ohne selbst beurteilen zu können, ob ein Fix tatsächlich greift. Das eigene Denken schaltet bei diesem Vorgehen zunehmend ab, man wartet nur noch passiv darauf, dass die KI etwas „macht“. Ein Entwicklungsprozess, der sich als absolut langweilig erwies. Kleine, im Detail kaum nachvollziehbare Fehler führten dabei wiederholt dazu, dass die Software insgesamt

nicht mehr funktionierte, ohne dass sich als Anwender noch klären ließ, woran es tatsächlich scheiterte.

Der Agent erledigte Aufgaben zudem häufig nur zur Hälfte: Mal vergaß er, zugehörige Tests anzupassen, mal ließ sich das Projekt anschließend gar nicht mehr kompilieren, weil schlicht Imports fehlten. Selbst bei expliziten Prompts wie „Behebe Compiler-Fehler und Tests“ nahm er zwar Änderungen vor, ohne dass diese jedoch tatsächlich funktionierten, stattdessen traten neue oder andere Fehler auf. Viele Aspekte, die nicht bis ins Detail spezifiziert waren oder dem Agenten schlicht nicht im Kontext vorlagen, obwohl sie bereits durch andere Microservices vorgegeben waren, wurden dadurch falsch implementiert oder fälschlich angenommen. Auch der automatisierte GitHub-Reviewer erwies sich als problematisch: Einmal wurden dessen Vorschläge übernommen, woraufhin die Anwendung anschließend gar nicht mehr funktionierte.

Auch wirtschaftlich zeigten sich deutliche Schattenseiten. Ohne das kostenfreie Abonnement von Google hätte das Budget nicht gereicht. Ein einzelner Agenten-Durchlauf für einen Microservice verbrauchte bereits mehr als die Hälfte des ursprünglich veranschlagten Budgets von rund 50 €. Während des Projekts wurde mehr als 500-mal deployt; rechnet man nun das Budget hoch, kommt man auf astronomische Kosten. Gelernt hat das Team bei diesem Vorgehen letztlich kaum etwas. Verschärft wurde diese Situation durch eine hochgradig unübersichtliche Werkzeuglandschaft. Es existiert eine kaum überschaubare Zahl an Agentensystemen, die im Grunde denselben Zweck verfolgen. Claude Code, Codex, Antigravity, GitHub Copilot, Cursor oder OpenCode, jeweils sowohl als CLI- als auch als Desktop-Variante, jedes mit eigener Bedienung und Konfiguration, während zugleich in kurzen Abständen neue Tools und Versionen erscheinen und die aktive Nutzung häufig kostenpflichtige Abonnements voraussetzt. Mit einer Testabdeckung von über 80 % erreichte das Projekt zwar einen auf den ersten Blick guten Wert, doch lässt sich daraus kaum etwas über die tatsächliche Qualität der Implementierung oder der Tests selbst ableiten. Wie verlässlich die Anwendung letztlich wirklich getestet ist, bleibt offen.

5.3 Herausforderungen

Die wirtschaftliche Nutzung des Vibe Codings wird oft mit einer drastischen Beschleunigung der Entwicklungszyklen begründet. Der finanzielle Aufwand für den Einsatz von Werkzeugen wie Claude Code hat sich von den ursprünglichen Lizenzgebühren einfacher Abonnements (20 US-Dollar pro Monat) hin zu erheblichen Token-Kosten entwickelt. Kosten für Teams, die Werkzeuge der KI nutzen, liegen heute eher zwischen 200 und teils mehr als 2.000 US-Dollar pro Entwickler und Monat [16]. In Einzelfällen gibt es bereits Berichte von Mitarbeitern, die über 1,4 Millionen US-Dollar pro Monat verbrauchten [17]. Diese Investitionen führen durchaus zu einer höheren Anzahl an geschriebenen

Zeilen Quellcode, doch sagt diese Metrik nichts über den tatsächlichen Nutzen aus. „Anybody who thinks that’s a valid metric is too stupid to work at a tech company.“ - Linus Torvalds (über Zeilen von Code als KPI) [18].

Eine der umfangreichsten empirischen Untersuchungen zu diesem Phänomen ist die Langzeitstudie des Analyseunternehmens GitClear, die über einen Zeitraum von fünf Jahren die strukturellen Veränderungen von 211 Millionen Zeilen Quellcode aus Repositories führender Technologiekonzerne wie Google, Microsoft und Meta sowie globaler Großkonzerne auswertete. Die Ergebnisse zeigen eine Beschleunigung in qualitativer Degradierung moderner Codebasen [19], [20].

Der wohl folgenschwerste negative Effekt des Vibe Codings betrifft nicht die technische Beschaffenheit des Codes selbst, sondern die intellektuelle Kapazität des Entwicklerteams. Während technische Schulden ein im Code verankertes Phänomen beschreiben, das durch Refactoring behoben werden kann, entsteht durch den massiven Einsatz generativer Werkzeuge eine kognitive Verschuldung (Cognitive Debt). Diese beschreibt den fortschreitenden Verlust des Systemverständnisses, der sogenannten „Theorie des Systems“ [21]. Eine viermonatige Studie des MIT Media Lab, die im August 2025 veröffentlicht wurde, konnte unter Einsatz von EEG verschiedenste negative Effekte des Einsatzes generativer KI nachweisen [22]. So zeigten Nutzer, die intensiv auf Unterstützung durch LLM zurückgriffen, die schwächste neuronale Konnektivität und die geringste Gehirnaktivität. Als die Unterstützung durch KI entzogen wurde und die Probanden Aufgaben wieder eigenständig lösen mussten, zeigten sie eine signifikant reduzierte Alpha- und Beta-Konnektivität. Dieser Zustand der neuronalen Unteraktivierung (Neural Under-Engagement) belegt, dass das Gehirn nach längerer Nutzung von KI massive Schwierigkeiten hat, sich wieder eigenständig und tiefgehend auf komplexe kognitive Prozesse einzustellen.

5.4 Zukunfts-Ausrichtung

Die praktische Evaluation im Rahmen dieses Projekts bestätigt das zentrale Versprechen des „Vibe Codings“: Für die rasche Umsetzung eines MVP oder eines PoC ermöglicht der Einsatz agentischer Systeme der KI eine beachtliche Entwicklungsgeschwindigkeit, selbst ohne tiefgehende Vorkenntnisse im jeweiligen Tech-Stack. Allerdings entpuppt sich dieser initiale Geschwindigkeitsvorteil bei genauerer Betrachtung als zweischneidiges Schwert. Wie unsere praktischen Erfahrungen und die empirischen Befunde der GitClear-Studie untermauern, erkaufte man sich die kurzfristige Produktivität häufig mit systematischen Qualitätsverlusten, inkonsistenten Implementierungen, unzureichend verifizierbaren Testabdeckungen und halluziniertem Code.

Weit schwerwiegender als diese rein technischen Defizite ist jedoch die schleichende Erosion der menschlichen Kompetenz. Die von uns empfundene Frustration bei der Fehlersuche und das passive „Abschalten des Gehirns“ während des Wartens auf den Agenten korrelieren präzise mit den neurobiologischen Erkenntnissen der Studie des MIT zur kognitiven Verschuldung (*Cognitive Debt*). Wenn Entwickler durch den kontinuierlichen Einsatz autonomer Agenten die „Theorie des Systems“ verlieren, entsteht bereits nach kurzen Projektphasen ein fataler Teufelskreis: Da nach der Erstellung des MVP niemand von uns mehr ein ganzheitliches Code-Verständnis aufweist, muss die Behebung von Fehlern oder Architekturproblemen zwangsläufig wieder an den Agenten der KI delegiert werden. Dies treibt nicht nur die von uns beobachteten Token-Kosten in astronomische Höhen, sondern manifestiert eine langfristige technologische und wirtschaftliche Abhängigkeit (*Vendor Lock-in*) von großen Anbietern von LLM. Sollte in Zukunft eine manuelle Überarbeitung des Codes erforderlich werden, beispielsweise durch weiter steigende Abonnements- und Tokenpreise oder komplexe Systemerweiterungen, ist eine menschliche Aufarbeitung der historisch gewachsenen Codebase kaum noch realisierbar und käme einer ressourcenintensiven Neuentwicklung gleich.

Für eine zukunftssichere und verantwortungsvolle Softwareentwicklung leitet sich daraus eine klare Differenzierung ab: Systeme der KI besitzen als unterstützende Werkzeuge eine enorme Daseinsberechtigung. Ihr Einsatz sollte jedoch bewusst und gezielt erfolgen, beispielsweise als interaktiver „Pair Programmer“ zur Beantwortung spezifischer Architekturfragen, zur Analyse von isolierten Fehlern oder zur Beschleunigung repetitiver Muster, bei denen das kritische Denken und die Entscheidungshoheit beim Menschen verbleiben. Das vollständige Überlassen des Entwicklungsprozesses an autonome Coding-Agenten erweist sich hingegen als nicht nachhaltig. Da Software im produktiven Unternehmensumfeld in der Regel über Jahrzehnte betrieben, gewartet und an neue Anforderungen angepasst werden muss, bleiben menschliche Experten, die das System in seiner tiefen Architektur durchdringen und beherrschen, auch in Zukunft unersetzlich.

6 Kurze Zusammenfassung

6.1 Zusammenfassung

Im Rahmen dieser Arbeit wurde mit PeerReview ein webbasiertes System zur gegenseitigen Begutachtung wissenschaftlicher Arbeiten als MVP realisiert. Der eigentliche Begutachtungsprozess, von der Konfiguration einer Abgabe über die Zuweisung eines Gutachters und die Einreichung der Arbeit bis zur Rückgabe des fertigen Gutachtens, wird von vier eng zusammenspielenden Kern-Services getragen, ergänzt um vier weitere Services für Kommunikation, Benachrichtigungen, Nutzerverwaltung und die Web-Oberfläche (siehe Abschnitt 3). Die in der Zielsetzung geforderte Plugin-Architektur wurde im Configuration Service umgesetzt und mit acht Plugins für unterschiedliche Review-Typen und Bewertungsformate ausgestattet, sodass sich neue Begutachtungsworkflows ohne Eingriff in den Kern des Systems ergänzen lassen.

Die gesamte Infrastruktur wurde vollständig über CDK deklariert und automatisiert auf serverlosen, containerisierten Ressourcen bereitgestellt. Ein durchgängiges Muster über alle Services hinweg bildet dabei die konsequente Trennung von Zuständigkeiten, etwa durch das Database-per-Service-Prinzip, klar abgegrenzte Autorisierungsebenen oder austauschbare Strategie-Implementierungen wie im Notification Service. Ergänzt wurde die Entwicklung durch eine GitHub-Actions-Pipeline für Build, Test und Deployment. Über den gesamten Entwicklungsprozess hinweg kamen zudem verschiedene agentische Coding-Werkzeuge wie Claude Code, Google Antigravity und OpenCode zum Einsatz. Deren Nutzung hatte jedoch ihren Preis, die Testabdeckung fiel zwischen den Services zunächst höchst uneinheitlich aus und blieb in Teilen, etwa in der Web-UI, zeitweise vollständig aus, weshalb sie erst nachträglich über den Index CRAP gezielt sichtbar gemacht und manuell nachgebessert werden musste (siehe Abschnitt 4.6). Nutzen und Grenzen des agentischen Vorgehens werden ausführlich in Abschnitt 4 sowie im Kapitel Abschnitt 5 diskutiert.

6.2 Fazit

Die in Abschnitt 1.2 formulierten technischen Ziele wurden zu einem großen Teil erreicht. Der Begutachtungsprozess ist als Menge unabhängiger, lose gekoppelter Microservices umgesetzt, die Plugin-Architektur erlaubt die Erweiterung um neue Review-Verfahren, die Infrastruktur liegt vollständig als Code vor und ist über eine automatisierte Pipeline ausrollbar, und sämtliche Schnittstellen sind über OpenAPI dokumentiert. Das System bildet damit den in der Aufgabenstellung geforderten Kernprozess funktionsfähig ab und lässt sich, wie in Abschnitt 4.6 beschrieben, agil um weitere Funktionalität ergänzen.

Differenzierter fällt unsere Bewertung des übergeordneten, experimentellen Ziels aus, zu untersuchen, wie weit sich eine verteilte Architektur mithilfe agentischer Coding-Werkzeuge realisieren lässt. Einerseits ermöglichte uns der Einsatz von Claude Code, Google Antigravity und weiteren Werkzeugen eine für ein Greenfield-Projekt dieser Größenordnung ungewöhnlich hohe Entwicklungsgeschwindigkeit und erlaubte auch weniger erfahrenen Mitgliedern unseres Teams, funktionsfähige Services beizutragen. Andererseits bestätigten sich über den Projektverlauf hinweg fast alle im Kapitel Abschnitt 5 festgehaltenen Vorbehalte. Die bereits erwähnte, erst nachträglich über den Index CRAP aufgedeckte Testabdeckungslücke ist dabei weniger als gewöhnliche Qualitätssicherungsmaßnahme zu verstehen, sondern als unmittelbarer Beleg dafür, dass sich die Agenten ohne wiederholte manuelle Nachsteuerung nicht durchgehend an unsere vereinbarten Konventionen hielten. Hinzu kamen unsere Erfahrungen mit einem spürbaren Verlust an Systemverständnis bis hin zur kognitiven Verschuldung sowie mit häufig nur halb umgesetzten Änderungen. Auch die finanzielle Dimension blieb nicht folgenlos: Bereits ein einzelner Agenten-Lauf für einen Microservice verbrauchte mehr als die Hälfte unseres ursprünglich veranschlagten Projektbudgets von rund 50 \$. Insgesamt lässt sich festhalten, dass agentisches Coding im Rahmen dieses Projekts für uns vor allem dann verlässlich funktionierte, wenn wir es durch klare architektonische Leitplanken wie das Monorepo, die initiale Baseline und den Template Service einhegten und durch wiederholte manuelle Kontrolle begleiteten. Ohne diese Kontrolle führte es schnell zu inkonsistenten oder sogar funktionsuntüchtigen Ergebnissen. Für die Entwicklung verteilter Systeme erscheint es uns damit derzeit eher als wirkungsvoller Beschleuniger unter durchgehender menschlicher Kontrolle denn als Ersatz für diese.

6.3 Ausblick

Fachlich wurden im Rahmen des MVP bewusst mehrere Funktionen zurückgestellt, die sich für eine Weiterentwicklung anbieten, darunter Annotationen in Dokumenten im Format PDF im Browser, die Unterstützung weiterer Einreichungsformate neben PDF, ein automatisierter Erinnerungsdienst für anstehende Fristen sowie ein eigenständiger Reporting-Service für komplexere statistische Auswertungen. Auf infrastruktureller Seite wurde aus Kostengründen bislang auf eine Architektur mit Multi-AZ verzichtet, ebenso auf eine dauerhafte Provisioned Concurrency für die eingesetzten Lambda-Proxies, wodurch bei geringer Last spürbare Kaltstart-Latenzen entstehen können. Für ein produktives Wachstum über den aktuellen Nutzungsumfang hinaus wäre außerdem ein Pub/Sub-Backend wie Redis erforderlich, um die aktuell instanzgebundenen SSE im Communication und Notification Service auch bei mehreren parallelen Task-Instanzen zuverlässig auszuliefern.

Prozessual legt die gemachte Erfahrung nahe, projektspezifische `AGENTS.md`-Dateien bereits vor dem eigentlichen Implementierungsbeginn deutlich detaillierter auszuarbeiten, statt Konventionen und Testabdeckung erst nachträglich über Maßnahmen wie die Optimierung des Index CRAP anzugleichen. Ein solches Vorgehen könnte die im Projektverlauf beobachtete Uneinheitlichkeit zwischen den agentisch entwickelten Services von Anfang an deutlich reduzieren.

A Repository

Der vollständige Quellcode, die Infrastrukturdefinitionen und die begleitende Dokumentation werden in einem Monorepository verwaltet. Das Repository ist öffentlich unter github.com/fh-wedel/MSA26-PeerReview-Gruppe-A verfügbar. Die folgende Übersicht beschreibt die wichtigsten Bereiche.

- **Anwendungscode:**

- `configuration-service/`, `matchingService/`, `submission-service/` und `responseService/` enthalten die fachlichen Kernservices für Konfiguration, Gutachterzuordnung, Einreichung und Ergebnisverwaltung.
- `communicationService/`, `notificationService/` und `userService/` enthalten die ergänzenden Services für Kommunikation, Benachrichtigungen und Nutzerverwaltung.
- `web-ui/` enthält die React-basierte Benutzeroberfläche.
- `api-client/` enthält gemeinsam verwendete Java-Komponenten für die Backend-Services.
- `templateEcsService/` dient als Vorlage für die Anlage weiterer Services.

- **Infrastruktur und Deployment:**

- `infrabaseline/` enthält die grundlegende Infrastruktur auf AWS, die vor den Services bereitgestellt wird.
- `infraLibrary/` enthält wiederverwendbare Bausteine für CDK zur Service-Infrastruktur.
- `cloudfront/` enthält die zentrale Routing- und Auslieferungskonfiguration.
- `.github/` enthält die GitHub-Actions-Workflows für Build, Tests und Deployment.

- **Dokumentation und Tests:**

- `Paper/` enthält den Projektbericht als Typst-Projekt inklusive Abbildungen und Literaturverzeichnis.
- `doc/` enthält Aufgabenstellung und Architekturdiagramme
- `postman/` enthält Collections und Hilfsdateien für manuelle Tests der API.
- `Präsentation/` enthält die Präsentationsfolien.

- **Entwicklungsunterstützung:**

- `AGENTS.md` und weitere gleichnamige Dateien enthalten projektspezifische Hinweise für Coding-Agenten.
- `.opencode/` enthält die Konfiguration der verwendeten OpenCode-Agenten.

Literaturverzeichnis

- [1] „AGENTS.md: A standard for AI coding agents instructions“. Zugegriffen: 2. Juli 2026. [Online]. Verfügbar unter: <https://agents.md/>
- [2] DylanLlivi, „Feature Request: Support AGENTS.md“. Zugegriffen: 2. Juli 2026. [Online]. Verfügbar unter: <https://github.com/anthropics/claude-code/issues/6235>
- [3] „Google Antigravity“. Zugegriffen: 4. Juli 2026. [Online]. Verfügbar unter: <https://antigravity.google/>
- [4] „Antigravity IDE“. Zugegriffen: 4. Juli 2026. [Online]. Verfügbar unter: <https://antigravity.google/product/antigravity-ide>
- [5] „Visual Studio Code - The open source AI code editor“. Zugegriffen: 4. Juli 2026. [Online]. Verfügbar unter: <https://code.visualstudio.com/>
- [6] „Antigravity 2.0“. Zugegriffen: 4. Juli 2026. [Online]. Verfügbar unter: <https://antigravity.google/product/antigravity-2>
- [7] „Antigravity CLI“. Zugegriffen: 4. Juli 2026. [Online]. Verfügbar unter: <https://antigravity.google/product/antigravity-cli>
- [8] „Claude Code“. Zugegriffen: 18. Juli 2026. [Online]. Verfügbar unter: <https://www.claude.com/product/claude-code>
- [9] N. J. Engelking, „Anthropic is removing OpenClaw from its Claude subscriptions“. Zugegriffen: 1. Juli 2026. [Online]. Verfügbar unter: <https://www.heise.de/en/news/Anthropic-is-removing-OpenClaw-from-its-Claude-subscriptions-11246096.html>
- [10] „Using GitHub Copilot code review“. Zugegriffen: 4. Juli 2026. [Online]. Verfügbar unter: <https://docs.github.com/en/copilot/how-tos/use-copilot-agents/request-a-code-review/use-code-review>
- [11] „OpenAPI Initiative“. Zugegriffen: 12. Juli 2026. [Online]. Verfügbar unter: <https://www.openapis.org/>
- [12] „Postman: The World's Leading API Platform“. Zugegriffen: 12. Juli 2026. [Online]. Verfügbar unter: <https://www.postman.com/>
- [13] „draw.io: Security-first diagramming for teams“. Zugegriffen: 12. Juli 2026. [Online]. Verfügbar unter: <https://www.drawio.com/>

- [14] „Typst: The new foundation for documents“. Zugegriffen: 12. Juli 2026. [Online]. Verfügbar unter: <https://typst.app/>
- [15] „What is Vibe Coding?“. Zugegriffen: 4. Juli 2026. [Online]. Verfügbar unter: <https://www.ibm.com/think/topics/vibe-coding>
- [16] A. Kanitkar, „Developer Productivity Benchmarks 2026: AI-Native Engineering Data“. Zugegriffen: 4. Juli 2026. [Online]. Verfügbar unter: <https://larridin.com/developer-productivity-hub/developer-productivity-benchmarks-2026>
- [17] J. Munis, „A Meta employee created a dashboard so coworkers can compete to be the company's No. 1 AI token user—and Zuckerberg doesn't even rank in the top 250“. Zugegriffen: 4. Juli 2026. [Online]. Verfügbar unter: <https://fortune.com/2026/04/09/meta-killed-employee-ai-token-dashboard/>
- [18] „Building the PERFECT Linux PC with Linus Torvalds (Zeit 37:00 - 37:25)“. Zugegriffen: 4. Juli 2026. [Online]. Verfügbar unter: <https://youtu.be/mfv0V1SxbNA?si=194GnFk2K2L2lZ8-&t=2220>
- [19] „AI Copilot Code Quality: 2025 Look Back at 12 Months of Data“. Zugegriffen: 4. Juli 2026. [Online]. Verfügbar unter: https://www.gitclear.com/ai_assistant_code_quality_2025_research
- [20] B. Harding, „GitClear AI Code Quality Research Pre-Release“. Zugegriffen: 4. Juli 2026. [Online]. Verfügbar unter: https://www.gitclear.com/blog/gitclear_ai_code_quality_research_pre_release
- [21] M.-A. Storey, „Cognitive debt: The hidden risk in AI-driven software development“. Zugegriffen: 4. Juli 2026. [Online]. Verfügbar unter: <https://getdx.com/blog/cognitive-debt-the-hidden-risk-in-ai-driven-software-development/>
- [22] N. Kosmyna *u. a.*, „Your Brain on ChatGPT: Accumulation of Cognitive Debt when Using an AI Assistant for Essay Writing Task“. Zugegriffen: 4. Juli 2026. [Online]. Verfügbar unter: <https://www.media.mit.edu/publications/your-brain-on-chatgpt/>

Eigenständigkeitserklärung

Ich erkläre hiermit an Eides statt, dass ich die vorliegende Arbeit selbstständig und ohne Benutzung anderer als der angegebenen Hilfsmittel angefertigt habe. Die aus fremden Quellen direkt oder indirekt übernommenen Gedanken sind als solche kenntlich gemacht.

Darüber hinaus erkläre ich, dass die im Rahmen dieser Arbeit erstellte Software anforderungsgemäß unter Verwendung von Werkzeugen der künstlichen Intelligenz (KI) entwickelt wurde. Zudem wurden KI-Werkzeuge zur Grammatikkorrektur, zur Identifikation von Synonymen, zur Übersetzung einzelner Begriffe, zum Verständnis von Konzepten sowie zur Verbesserung der Lesbarkeit eingesetzt.

Die Arbeit wurde bisher in gleicher oder ähnlicher Form keiner anderen Prüfungskommission vorgelegt und auch nicht veröffentlicht.



Matthias Matthies



Marcel Ossig



Luca Jannsen



Gideon Gyebi